

Universidade Lusófona

Departamento de Engenharia Informática e Sistemas de Informação (DEISI)

Uma perspectiva de engenharia sobre a evolução de algoritmos de ranqueamento baseada em grafos

*Comparação empírica de PageRank, HITS e SALSA sobre grafos de
requisitos extraídos por análise estática*

Tales Santos

Dissertação para obtenção do Grau de Mestre em

Mestrado em Engenharia Informática e Sistemas de Informação

Orientador: Aleksandar Mikovic — *Professor, Universidade Lusófona*
Coorientadora: Sofia Fernandes — *Professora, Universidade Lusófona*

Lisboa, Setembro de 2026

Revisão R6

Agradecimentos

Espaço reservado para os agradecimentos. Recomenda-se reconhecer o trabalho dos orientadores, do júri e das pessoas e instituições que contribuíram de forma relevante para a elaboração da dissertação.

Resumo

A priorização de requisitos de software é uma das atividades mais sensíveis da Engenharia de Requisitos: técnicas tradicionais como a **Votação Acumulativa** e o **PERT/CPM** — descritas em detalhe no Capítulo 2 — dependem fortemente da percepção subjetiva dos *stakeholders* e tendem a tornar-se inconsistentes em sistemas com elevado número de requisitos interdependentes (Karlsson, 2002; Silva et al., 2018). Esta dissertação investiga uma alternativa de natureza estrutural, assente na aplicação de algoritmos de ranqueamento de grafos — PageRank, HITS e SALSA — ao grafo de dependências entre requisitos.

Foi desenvolvido o **ReqGraph**, um protótipo *open source* em Python que automatiza o *pipeline* *Código-fonte* → *AST* → *Call Graph* → *Grafo de Requisitos*, e o módulo `ranker.py`, que aplica os três algoritmos ao grafo extraído. A validação foi conduzida sobre três projetos reais de código aberto — *CPython stdlib*, *Flask* e *scikit-learn* — selecionados por representarem regimes crescentes de acoplamento arquitetural. A comparação entre os rankings produzidos pelos três algoritmos é feita de forma quantitativa, recorrendo aos coeficientes de **correlação de Spearman** (ρ) e **Kendall** (τ_b). Os resultados mostram que, em grafos com densidade suficiente, os algoritmos partilham parcialmente a identificação dos requisitos mais centrais; em grafos esparsos, divergem de forma informativa, oferecendo perspectivas complementares de importância — transitiva (PageRank), direta (HITS *authority*) e de orquestração (HITS *hub*). Conclui-se que o ranqueamento estrutural constitui um complemento robusto às técnicas tradicionais, contribuindo para a redução — não a eliminação — do viés subjetivo na priorização de requisitos. Discutem-se em detalhe as limitações da abordagem, nomeadamente a subjetividade residual do mapeamento `func_to_req` e a dimensão reduzida do corpus.

Palavras-chave: Engenharia de Requisitos; Priorização Estrutural; Análise Estática de Código; Grafos; PageRank; HITS; SALSA; Correlação de Spearman.

Abstract

Software requirements prioritization is one of the most sensitive activities in Requirements Engineering. Traditional techniques such as **Cumulative Voting** and **PERT/CPM** — described in detail in Chapter 2 — rely heavily on the subjective perception of stakeholders and tend to become inconsistent in systems with a large number of interdependent requirements (Karlsson, 2002; Silva et al., 2018). This thesis investigates a structural alternative based on the application of graph ranking algorithms — PageRank, HITS, and SALSA — to the dependency graph between requirements.

A Python *open-source* prototype, **ReqGraph**, was developed to automate the pipeline *source code* $\rightarrow$ *AST* $\rightarrow$ *call graph* $\rightarrow$ *requirements graph*, together with the `ranker.py` module which applies the three algorithms to the extracted graph. Validation was conducted on three real open-source projects — *CPython stdlib*, *Flask*, and *scikit-learn* — selected as representatives of increasing regimes of architectural coupling. The agreement between the rankings produced by the three algorithms is quantified using the **Spearman** (ρ) and **Kendall** (τ_b) rank correlation coefficients. Results show that, in graphs with sufficient density, the algorithms partially agree on the most central requirements; in sparse graphs, they informatively diverge, offering complementary perspectives on importance — transitive (PageRank), direct (HITS *authority*), and orchestration (HITS *hub*). We conclude that structural ranking constitutes a robust complement to traditional techniques, reducing — but not eliminating — subjective bias in requirements prioritization. We discuss in detail the limitations of the approach, namely the residual subjectivity of the `func_to_req` mapping and the small corpus size.

Keywords: Requirements Engineering; Structural Prioritization; Static Code Analysis; Graphs; PageRank; HITS; SALSA; Spearman Correlation.

Índice

Agradecimentos	iii
Resumo	v
Abstract	vii
Abreviaturas, Siglas e Símbolos	xvii
1 Introdução	1
1.1 Enquadramento e Motivação	1
1.2 Problema	2
1.3 Objetivos e <i>Research Questions</i>	2
1.4 Contribuições	3
1.5 Estrutura do Documento	4
2 Conceitos Teóricos e Contextualização do Problema	5
2.1 Conceitos básicos de Grafos	5
2.1.1 Tipos de Grafos	6
2.1.2 Densidade: grafos esparsos e densos	7
2.1.3 Caminhos e passeios aleatórios	7
2.2 Algoritmos de ranqueamento em grafos	8
2.2.1 Centralidade de grau	8
2.2.2 PageRank	9
2.2.3 HITS (<i>Hyperlink-Induced Topic Search</i>)	9
2.2.4 SALSA (<i>Stochastic Approach for Link-Structure Analysis</i>)	10
2.2.5 BiRank e normalizações simétricas	11
2.2.6 Comparação de rankings: correlação de Spearman e Kendall	12
2.3 Análise estática e Árvore de Sintaxe Abstrata (AST)	12
2.3.1 O que é uma AST	13
2.3.2 Da AST ao Grafo de Requisitos: validação por fase	13
2.4 Aplicação das Métricas ao Ranking e Priorização de Requisitos	14

2.4.1	Fundamentos da priorização de requisitos e desafios	14
2.4.2	Construção do grafo de requisitos e posicionamento face a Silva et al. (2018)	15
2.4.3	Mapeamento dos algoritmos de ranqueamento para priorização	15
2.4.4	Benefícios, limitações e considerações práticas	16
3	Metodologia	17
3.1	Estratégia de Investigação	17
3.2	Arquitetura do Protótipo ReqGraph	18
3.3	Implementação e Tecnologias	19
3.3.1	Módulo de Ranqueamento (<code>ranker.py</code>)	20
3.3.2	PageRank	20
3.3.3	HITS	20
3.3.4	SALSA	20
3.3.5	Artefactos Gerados	21
3.4	Métricas de Comparação de Rankings	21
3.5	Corpus Experimental e Protocolo de Avaliação	22
3.5.1	Seleção dos Casos de Estudo	22
3.5.2	Protocolo de Construção do Mapeamento <code>func_to_req</code>	22
3.5.3	Protocolo Experimental	23
3.5.4	Dimensões de Análise	23
4	Avaliação de Resultados	25
4.1	Resultados Técnicos	25
4.1.1	Síntese Quantitativa	25
4.1.2	Caso Simples — CPython <i>stdlib</i>	26
4.1.3	Caso Intermediário — Flask	27
4.1.4	Caso Complexo — <i>scikit-learn</i>	28
4.2	Rankings produzidos pelos três algoritmos	29
4.2.1	Caso Simples — Rankings para o CPython <i>stdlib</i>	29
4.2.2	Caso Intermediário — Rankings para o Flask	30
4.2.3	Caso Complexo — Rankings para <i>scikit-learn</i>	31
4.3	Correlações Spearman e Kendall entre Rankings	31
4.4	Comparação Consolidada	33
4.5	Discussão	33

4.6	Ameaças à Validade	34
4.6.1	Validade de Construto (<i>Construct Validity</i>)	35
4.6.2	Validade Interna (<i>Internal Validity</i>)	35
4.6.3	Validade Externa (<i>External Validity</i>)	35
4.6.4	Validade de Conclusão (<i>Conclusion Validity</i>)	36
5	Conclusão	37
5.1	Sumário e Principais Contribuições	37
5.2	Trabalho Futuro	39
	Bibliografia	41
	Glossário	43
	Glossário	43
	Apêndice ou Anexo?	45
	Cálculo de Spearman e Kendall em Python puro	47
	Mapeamentos <code>func_to_req</code> dos três casos de estudo	49

Lista de Figuras

2.1	Grafo exemplo (rede de colaboração entre investigadores).	6
2.2	Tipos de grafos discutidos na Secção 2.1.1.	7
2.3	Grafo exemplo usado para ilustrar o PageRank.	9
2.4	Grafo exemplo usado para ilustrar o HITS.	10
2.5	Grafo exemplo usado para ilustrar o SALSA: os <i>ranks</i> são proporcionais aos graus de entrada (<i>authority</i>) e de saída (<i>hub</i>).	11
3.1	<i>Call Graph</i> extraído via AST do CPython <i>stdlib</i> (<i>argparse</i> + <i>http.server</i>). Cada vértice é uma função ou método; cada aresta (f_i, f_j) indica que f_i invoca f_j . . .	19
4.1	Grafo de Requisitos: CPython <i>stdlib</i> . Note-se a divisão clara em dois sub-sistemas independentes (<i>parsing/argparse</i> vs. HTTP) e o ciclo <code>FORMATTING</code> $\leftrightarrow$ <code>ERROR_HANDLING</code>	27
4.2	Grafo de Requisitos: Flask. O ciclo <code>REQUEST_HANDLING</code> $\leftrightarrow$ <code>CONTEXT</code> $\leftrightarrow$ <code>ROUTING</code> $\leftrightarrow$ <code>ERROR_HANDLING</code> evidencia o forte acoplamento típico do <i>middleware</i>	28
4.3	Grafo de Requisitos: <i>scikit-learn</i> . O caso mais denso (17 arestas) reflete a orquestração interna do <i>framework</i> : <code>REQ_PIPELINE_FIT</code> e <code>REQ_PIPELINE_PREDICT</code> conectam-se cada um a cinco outros requisitos.	29
4.4	Rankings produzidos pelos três algoritmos para o caso CPython <i>stdlib</i>	30
4.5	Rankings produzidos pelos três algoritmos para o caso Flask.	31
4.6	Rankings produzidos pelos três algoritmos para o caso <i>scikit-learn</i>	32
4.7	Ranking comparativo entre os três projetos: <i>top-1</i> por algoritmo em cada caso de estudo. Compare-se com as Tabelas 4.2, 4.3 e 4.4.	33

Lista de Tabelas

3.1	Tecnologias utilizadas e respectivas funções no <i>pipeline</i>	19
3.2	Projetos selecionados como casos de estudo.	22
4.1	Síntese quantitativa dos três casos de estudo. <i>Densidade</i> é calculada como $ E /(n(n-1))$ sobre requisitos com arestas.	26
4.2	<i>Top-3</i> por algoritmo: CPython <i>stdlib</i>	29
4.3	<i>Top-3</i> por algoritmo: Flask.	30
4.4	<i>Top-3</i> por algoritmo: <i>scikit-learn</i>	31
4.5	Coefficientes de correlação de ranks (Spearman ρ , Kendall τ_b) entre cada par de algoritmos, por caso de estudo. <i>Top-1</i> indica se o requisito <i>top-1</i> coincide.	32
5.1	Estado dos objetivos e das <i>research questions</i>	38

Abreviaturas, Siglas e Símbolos

Símbolo / Sigla	Descrição
AHP	<i>Analytic Hierarchy Process</i> (Saaty, 1980)
AST	<i>Abstract Syntax Tree</i>
COFAC	Cooperativa de Formação e Animação Cultural
CPM	<i>Critical Path Method</i>
DEISI	Departamento de Engenharia Informática e Sistemas de Informação
ER	Engenharia de Requisitos
HITS	<i>Hyperlink-Induced Topic Search</i>
NDCG	<i>Normalized Discounted Cumulative Gain</i>
PERT	<i>Program Evaluation and Review Technique</i>
PR	PageRank (no contexto de algoritmo) ou Priorização de Requisitos
PRS	Priorização de Requisitos de Software
RE	<i>Requirements Engineering</i>
RP	<i>Requirements Prioritization</i>
RQ	<i>Research Question</i>
SALSA	<i>Stochastic Approach for Link-Structure Analysis</i>
SE	<i>Software Engineering</i>
TKC	<i>Tightly Knit Community</i>
$G = (V, E)$	Grafo com conjunto de vértices V e conjunto de arestas E
$G_R = (V_R, E_R)$	Grafo de Requisitos
$\mathbf{r}$	Vetor de <i>ranks</i> de um algoritmo
d	Fator de amortecimento do PageRank ($d = 0,85$ por convenção)
M	Matriz de transição estocástica
$\mathbf{e}$	Vetor de personalização do PageRank ($e_i = 1/n$ na variante uniforme)
A	Matriz de adjacência
$A_{ij} = 1$	Existe aresta de j para i (em grafos direcionados)
$\text{in}(v), \text{out}(v)$	Grau de entrada / grau de saída do vértice v
ρ (Spearman)	Coefficiente de correlação de ranks de Spearman
τ_b (Kendall)	Coefficiente de correlação de ranks de Kendall com correção para <i>ties</i>

Capítulo 1

Introdução

O Capítulo 1 enquadra o problema da priorização estrutural de requisitos. A Secção 1.1 discute a motivação e os fatores recentes que tornaram esta investigação oportuna; a Secção 1.2 define formalmente o problema; a Secção 1.3 enuncia os objetivos e as *research questions* do trabalho; a Secção 1.4 lista as contribuições; e a Secção 1.5 apresenta a estrutura do restante documento.

Conceitos-chave: enquadramento; motivação; problema; priorização; *research questions*; contribuições.

1.1 Enquadramento e Motivação

O desenvolvimento de sistemas de software em larga escala enfrenta desafios crescentes na gestão da complexidade e na organização de grandes volumes de informação interdependente (Sommerville, 2007). Tradicionalmente, a relevância de elementos dentro de uma rede de requisitos era determinada por análises textuais, comparações entre pares ou heurísticas manuais (Berander & Andrews, 2005; Karlsson & Ryan, 1997; Karlsson et al., 2007); a evolução da **Teoria dos Grafos** (Newman, 2010; Szwarcfiter, 2018) permitiu uma transição para métodos puramente estruturais, capazes de extrair informação latente diretamente da topologia do sistema.

A motivação principal para este estudo surge da convergência entre algoritmos de alta performance já em produção na indústria e a necessidade de rigor analítico na Engenharia de Software. Dois marcos fundamentais impulsionaram esta investigação:

- **Adopção industrial do SALSA pelo Twitter.** Em *Who To Follow*, o sistema de recomendação de utilizadores do Twitter, Gupta et al. (2013) propõem e descrevem uma variante do algoritmo **SALSA** (Lempel & Moran, 2001) para sugerir conexões entre utilizadores. Esta adoção em produção, sobre um grafo de larga escala, demonstra a viabilidade de usar passeios aleatórios em grafos para determinar prestígio e relevância de forma escalável.
- **Aplicação do PageRank à priorização de requisitos.** A descoberta de pesquisas que aplicam o **PageRank** (Page et al., 1999) à priorização de requisitos de software (Jin et al., 2009; Li & Yi, 2009; Silva et al., 2018) sustenta a hipótese de que a estrutura de dependências de um sistema contém informação latente suficiente para ordenar requisitos de forma mais consistente do que técnicas predominantemente subjetivas (Achimugu et al., 2014; Firesmith, 2004).

Dessa forma, o trabalho enquadra-se na busca por modelos matemáticos que minimizem o viés hu-

mano em decisões críticas de engenharia, utilizando propriedades topológicas para extrair inteligência de grafos de software.

1.2 Problema

O **ranqueamento de requisitos** — termo correntemente designado na literatura por **priorização de requisitos** (Bukhsh et al., 2020; Firesmith, 2004; Karlsson, 2002; Silva et al., 2018) — consiste em produzir uma ordem total (ou parcial) sobre o conjunto de requisitos $R = \{r_1, r_2, \dots, r_n\}$ de um sistema, a partir de critérios que reflitam a importância relativa de cada requisito para a entrega de valor, a estabilidade arquitetural ou o risco do projeto. Formalmente, dado o conjunto R e uma relação de dependência $D \subseteq R \times R$, em que $(r_i, r_j) \in D$ se r_i depende de r_j , a tarefa de priorização procura uma função $\pi : R \rightarrow \mathbb{R}$ que atribua a cada requisito um *score* compatível com a sua centralidade na rede (R, D) .

Nota ontológica

No presente trabalho, “requisito” é uma **agregação de responsabilidades implementadas no código-fonte**, identificada via um mapeamento explícito `func_to_req` (Secção 3). Não corresponde, portanto, a um requisito funcional clássico no sentido de Sommerville (2007), mas serve como *proxy* de *domain feature* para efeitos de ranqueamento estrutural. Esta opção é discutida em detalhe na Secção 2.4.2 (posicionamento face à literatura) e nas ameaças à validade (Secção 4.6).

As técnicas tradicionais de priorização — Comparação em Pares (Karlsson et al., 2007), Votação Acumulativa (Ahl, 2005; Leffingwell & Widrig, 2003), AHP (*Analytic Hierarchy Process*) (Karlsson & Ryan, 1997; Saaty, 1980) e PERT/CPM (Kerzner, 2009) — sofrem de problemas intrínsecos de subjetividade e esforço (Firesmith, 2004). A in experiência dos *stakeholders*, a divergência na interpretação das escalas de prioridade e o foco excessivo num único ponto de vista resultam em prioridades inconsistentes e em “posições inválidas” (Silva et al., 2018). Em particular, a Votação Acumulativa, por depender excessivamente da decisão direta do *stakeholder*, não resolve a duplicidade envolvida em requisitos mutuamente dependentes (Silva et al., 2018).

Este trabalho justifica-se, portanto, ao investigar e comparar a eficácia do **PageRank**, **HITS** e **SALSA** na priorização estrutural de requisitos, sobre grafos extraídos automaticamente do código-fonte, oferecendo um modelo analítico que minimize o esforço dos *stakeholders* e que utilize integralmente a rastreabilidade de requisitos disponível (Gotel & Finkelstein, 1994; Rasiman et al., 2022).

1.3 Objetivos e Research Questions

O objetivo desta dissertação é **investigar e comparar a eficácia** dos algoritmos PageRank, HITS e SALSA na priorização estrutural de requisitos de software, fornecendo um modelo analítico que utilize a rastreabilidade entre requisitos para garantir consistência em sistemas de larga escala. Em concreto, pretende-se: (i) **formalizar** a aplicação de processos estocásticos e de abordagens de Teoria dos Gra-

fos à modelação de interdependências entre requisitos e à determinação de fluxos de influência; (ii) **desenvolver** um protótipo (ReqGraph) que extraia, de forma totalmente automatizada e reproduzível, o grafo de requisitos a partir do código-fonte de um projeto Python; (iii) **avaliar empiricamente**, por meio de experimentação sobre projetos reais de código aberto, o comportamento comparativo dos três algoritmos, identificando convergências e divergências de ranking e discutindo-as à luz da arquitetura conhecida dos sistemas analisados.

Estes objetivos materializam-se nas seguintes *research questions* (RQs), formuladas de forma verificável e respondidas no Capítulo 4:

- RQ1.** Em que medida os rankings produzidos por PageRank, HITS e SALSA *concordam* entre si quando aplicados a grafos de requisitos extraídos automaticamente por análise estática (AST)? A concordância é medida pelo coeficiente de Spearman (ρ) e Kendall (τ_b) entre cada par de algoritmos.
- RQ2.** Como varia essa concordância em função das propriedades estruturais do grafo (densidade, presença de ciclos, simetria de graus)?
- RQ3.** Em que medida os *top-k requisitos* apontados pelos algoritmos correspondem a elementos conhecidos como arquiteturalmente centrais nos projetos analisados (i.e. qual a coerência arquitetural dos rankings produzidos)?

A formulação destas RQs permite distinguir o que esta investigação *demonstra* (acordo/desacordo entre algoritmos sobre os mesmos dados) do que *não* demonstra (validade preditiva face a juízos humanos — ver Secção 4.6 e Secção 5.2).

1.4 Contribuições

As principais contribuições desta dissertação são:

1. **Pipeline reproduzível para extração de grafos de requisitos.** Implementação do protótipo *ReqGraph*, que automatiza a transformação *Código-fonte* $\rightarrow$ *AST* $\rightarrow$ *Call Graph* $\rightarrow$ *Grafo de Requisitos* sobre projetos Python, recorrendo exclusivamente a análise estática e a artefactos canónicos da comunidade científica (NetworkX, Graphviz).
2. **Implementação aberta do SALSA.** Implementação manual e documentada do algoritmo SALSA (Lempel & Moran, 2001), uma vez que o NetworkX não a disponibiliza nativamente, incluindo o tratamento explícito de vértices absorventes por redistribuição uniforme.
3. **Comparação empírica quantitativa entre PageRank, HITS e SALSA** sobre grafos reais de requisitos derivados de três projetos *open source* representativos de regimes crescentes de acooplamento arquitetural (CPython *stdlib*, Flask e *scikit-learn*). A comparação utiliza métricas formais de correlação de ranks (Spearman ρ , Kendall τ_b) entre cada par de algoritmos.
4. **Mecanismo de redução do custo de mapeamento.** Formalização de um *prompt* estruturado (`llm_prompt_mapping.md`) que permite gerar o dicionário `func_to_req` via modelos

de linguagem de grande escala, mitigando a principal fricção operacional da abordagem.

O código-fonte completo, os mapeamentos `func_to_req` utilizados e os artefactos gerados nas experiências encontram-se publicamente disponíveis em https://github.com/tales96323/Tales_Tese_ulusofona.

1.5 Estrutura do Documento

O restante documento está organizado da seguinte forma. O **Capítulo 2** apresenta os conceitos teóricos relevantes — Teoria dos Grafos, algoritmos de ranqueamento e fundamentos da Engenharia de Requisitos — e contextualiza o problema, posicionando esta dissertação em relação ao trabalho de Silva et al. (2018), do qual constitui uma evolução metodológica. O **Capítulo 3** descreve a metodologia adotada, a arquitetura do protótipo ReqGraph, o protocolo experimental e o protocolo de mapeamento `func_to_req`. O **Capítulo 4** apresenta e discute os resultados obtidos nos três casos de estudo, recorrendo a métricas formais de correlação de ranks e a uma análise dedicada de *ameaças à validade* segundo o referencial de Wohlin et al. (2012). O **Capítulo 5** conclui o trabalho, sintetiza as contribuições e identifica linhas de trabalho futuro.

Capítulo 2

Conceitos Teóricos e Contextualização do Problema

O Capítulo 2 introduz os conceitos teóricos sobre os quais a dissertação assenta. A Secção 2.1 revisita conceitos basilares de Teoria dos Grafos, incluindo a noção de **densidade** (grafos esparsos e densos). A Secção 2.2 apresenta os algoritmos de ranqueamento utilizados, sublinhando que todas as métricas aqui consideradas são analisadas no contexto de **redes direcionadas**, e introduz os coeficientes de correlação de **Spearman** e **Kendall** que servem de critério de avaliação dos resultados. A Secção 2.3 apresenta a análise estática via Árvores de Sintaxe Abstrata (**AST**), técnica que sustenta a extração automática do grafo a partir do código-fonte e que fundamenta a validação do *pipeline* fase a fase. A Secção 2.4 contextualiza o problema da priorização de requisitos, posiciona este trabalho em relação à literatura existente — em particular, em relação a Silva et al. (2018) — e identifica a contribuição incremental da presente dissertação.

Conceitos-chave: grafos; densidade; grafos esparsos e densos; passeios aleatórios; PageRank; HITS; SALSA; correlação de ranks (Spearman, Kendall); análise estática; AST; priorização de requisitos; rastreabilidade.

2.1 Conceitos básicos de Grafos

Um grafo é uma estrutura matemática que modela relações entre objetos (Newman, 2010; Szwarcfiter, 2018). Formalmente, um grafo $G = (V, E)$ é definido por um conjunto de vértices (ou nós) V e um conjunto de arestas E , onde cada aresta é um par não ordenado (ou ordenado) de vértices. Os vértices representam as entidades de interesse, e as arestas representam as relações entre elas. Em redes complexas, os termos “grafo” e “rede” são frequentemente usados como sinónimos (Newman, 2010).

Exemplo. Considere uma pequena rede de colaboração entre investigadores. Os vértices podem ser $\{A, B, C, D\}$. As arestas representam coautorias: se A e B publicaram juntos, existe uma aresta entre eles. Suponha as colaborações $A-B$, $A-C$, $B-C$, $C-D$. O grafo correspondente tem $V = \{A, B, C, D\}$ e $E = \{AB, AC, BC, CD\}$.

Uma forma comum de representar um grafo é por meio da **matriz de adjacência** A , onde $A_{ij} = 1$

se existe uma aresta entre os vértices i e j , e 0 caso contrário (Newman, 2010). Para o exemplo acima:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

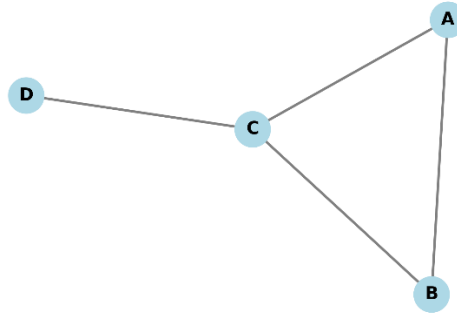


Figura 2.1 – Grafo exemplo (rede de colaboração entre investigadores).

2.1.1 Tipos de Grafos

Os grafos podem ser classificados de acordo com a **natureza dos vértices e das arestas** (Szwarcfiter, 2018). Os principais tipos relevantes para este trabalho são:

- **Grafos não direcionados:** As arestas não têm orientação, ou seja, a relação é simétrica. No exemplo anterior, a colaboração é mútua e, portanto, o grafo é não direcionado.
- **Grafos direcionados (digrafos):** As arestas possuem uma direção, indicando uma relação assimétrica. Por exemplo, em uma rede de citações, se o artigo A cita o artigo B , existe uma aresta de A para B , mas não necessariamente o contrário. Formalmente, as arestas são pares ordenados (u, v) . A matriz de adjacência, nesse caso, não é simétrica.
- **Grafos ponderados:** As arestas possuem um peso associado, que pode representar intensidade, custo ou distância. Por exemplo, em uma rede de transportes, o peso pode ser a distância entre cidades. A matriz de adjacência armazena os pesos: $A_{ij} = w_{ij}$.
- **Grafos bipartidos:** Os vértices podem ser particionados em dois conjuntos disjuntos U e V de tal forma que todas as arestas conectam um vértice de U a um vértice de V . Esse tipo de grafo é natural para modelar relações entre dois tipos distintos de entidades, como autores e artigos (Newman, 2010).
- **Grafos bipartidos ponderados:** Combinam as duas características anteriores: são grafos bipartidos cujas arestas possuem pesos. Um exemplo é uma rede de compras em que clientes (conjunto U) adquirem produtos (conjunto V) com determinada frequência (peso).

As classificações apresentadas seguem a sistematização clássica de Szwarcfiter (2018) e Newman

(2010); a Figura 2.2 ilustra os cinco tipos discutidos.

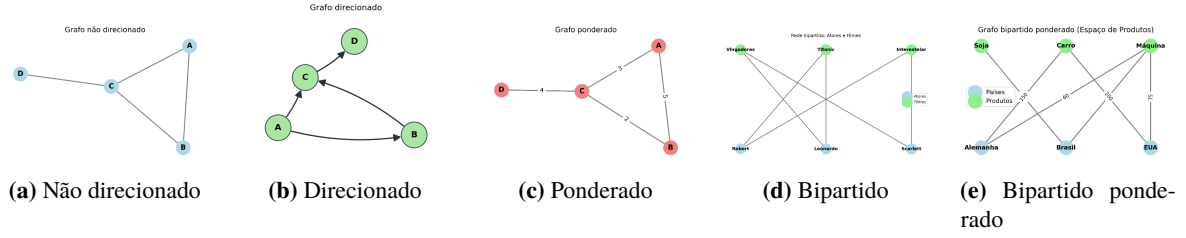


Figura 2.2 – Tipos de grafos discutidos na Secção 2.1.1.

2.1.2 Densidade: grafos esparsos e densos

Uma propriedade estrutural central de um grafo é a sua **densidade**, que mede a fração de arestas efetivamente presentes face ao número máximo possível (Newman, 2010). Para um grafo *direcionado* simples com $n = |V|$ vértices, o número máximo de arestas é $n(n - 1)$ — cada par ordenado de vértices distintos pode, quando muito, originar uma aresta —, pelo que a densidade se define como

$$D = \frac{|E|}{n(n - 1)} \in [0, 1]. \quad (2.1)$$

Para grafos não direcionados, o denominador é $n(n - 1)/2$, dado que cada aresta é não ordenada.

Diz-se que um grafo é **esparso** quando o número de arestas cresce aproximadamente de forma linear com n (densidade que tende para zero à medida que n aumenta) e **denso** quando $|E|$ se aproxima do máximo $\Theta(n^2)$ (densidade próxima de 1) (Newman, 2010; Szwarcfiter, 2018). A maioria dos grafos oriundos de sistemas reais — redes sociais, redes de citações ou grafos de dependências de software — é tipicamente *esparsa*: cada nó relaciona-se apenas com um pequeno subconjunto dos restantes.

Esta distinção é relevante para a presente dissertação por duas razões. Em primeiro lugar, a densidade é usada como *proxy* do grau de **acoplamento arquitetural** de cada projeto analisado: um Grafo de Requisitos mais denso indica responsabilidades mais interligadas (ver Capítulo 4, Secção 4.1, onde se reportam densidades de 0,125, 0,262 e 0,304 para os três casos de estudo, em ordem crescente de acoplamento). Em segundo lugar, em grafos de pequena dimensão — como os aqui estudados, com poucas dezenas de vértices — a densidade é uma métrica *volátil*: pequenas variações no número de arestas deslocam-na de forma significativa, pelo que deve ser interpretada em conjunto com outras métricas estruturais (presença de ciclos, *clustering*) e nunca isoladamente (Newman, 2010).

2.1.3 Caminhos e passeios aleatórios

Um caminho em um grafo é uma sequência de vértices $v_1, v_2, \dots, v_k$ tal que cada par consecutivo (v_i, v_{i+1}) é uma aresta. O comprimento do caminho é o número de arestas percorridas (Newman, 2010).

Um **passeio aleatório** (*random walk*) é um processo estocástico em que um “caminhante” se move ao longo do grafo de forma probabilística (Masuda et al., 2017; Ross, 2014). Em tempo discreto,

partindo de um vértice, o caminhante escolhe aleatoriamente uma das arestas incidentes e move-se para o vértice vizinho. Esse processo pode ser descrito por uma matriz de transição P , onde P_{ij} é a probabilidade de ir do vértice j para o vértice i . Para grafos não direcionados e não ponderados, uma escolha comum é $P_{ij} = 1/\text{grau}(j)$ se i e j são vizinhos.

Passeios aleatórios são a base de diversos algoritmos de ranqueamento, como o PageRank, que estima a importância de um vértice pela frequência com que é visitado em um passeio aleatório de longa duração (Masuda et al., 2017; Page et al., 1999).

2.2 Algoritmos de ranqueamento em grafos

O ranqueamento de vértices em grafos atribui um *score* a cada nó com base na estrutura de conexões. Diferentes algoritmos exploram diferentes propriedades — grau, centralidade de autovetor ou passeios aleatórios — e foram inicialmente propostos no contexto da análise de redes de citações, do *ranking* de páginas web e da análise de redes sociais (Kleinberg, 1999; Lempel & Moran, 2001; Page et al., 1999).

Importante

Todas as métricas descritas nesta secção são analisadas no contexto de **redes direcionadas**, dado que o grafo de requisitos é, por natureza, direcionado: a aresta $r_a \rightarrow r_b$ codifica a dependência de r_a sobre r_b .

Escolha de PageRank, HITS e SALSA face a alternativas. A literatura de *centrality measures* oferece um conjunto vasto de alternativas: *eigenvector centrality* (Newman, 2010), *Katz centrality*, *betweenness* e *closeness*, ou ainda decomposições *k-core*. Esta dissertação restringe a análise a PageRank, HITS e SALSA por três razões: (a) Silva et al. (Silva et al., 2018) estabelecem o PageRank como *baseline* no domínio — comparar com algoritmos de *mesma família* mantém o estudo *commensurate*; (b) HITS e SALSA introduzem a separação *hub/authority*, essencial para distinguir requisitos *orquestradores* de requisitos *críticos por dependência direta*; (c) as três métricas são interpretáveis como passeios aleatórios em cadeias de Markov, oferecendo uma fundamentação estocástica comum (Ding et al., 2002). A extensão a outras métricas (e.g. *Katz*, *k-core*) é discutida como trabalho futuro (Secção 5.2).

2.2.1 Centralidade de grau

A centralidade de grau é a métrica mais simples de centralidade em grafos (Newman, 2010). Para um vértice v , o **grau de entrada** (em grafos direcionados) é o número de arestas que chegam a ele, e o **grau de saída** é o número de arestas que partem. Formalmente, para um grafo com matriz de adjacência A , o grau de entrada de v é $\sum_u A_{uv}$. Esta métrica é a base do SALSA (Lempel & Moran, 2001), que na sua versão básica equivale a uma ponderação dos graus.

2.2.2 PageRank

O **PageRank** (PR) foi proposto por Page et al. (1999) e estima a importância de um vértice como a **distribuição estacionária** de um passeio aleatório em tempo discreto sobre o grafo (Brin & Page, 1998; Masuda et al., 2017). A intuição é que um vértice é importante se é apontado por outros vértices importantes — definição recursiva resolvida por iteração. O algoritmo incorpora um fator de amortecimento d (tipicamente 0,85), que representa a probabilidade de o caminhante continuar a seguir as arestas; com probabilidade $1 - d$, o caminhante salta para um vértice aleatório escolhido segundo uma distribuição de personalização (em geral, **uniforme** — todos os vértices têm idêntica probabilidade $1/n$ de serem escolhidos). A equação fundamental é:

$$\mathbf{r} = d M \mathbf{r} + (1 - d) \mathbf{e} \quad (2.2)$$

onde $\mathbf{r}$ é o vetor de *ranks*, M é a matriz de transição estocástica (cada coluna soma 1) e $\mathbf{e}$ é o vetor de personalização (uniforme, $e_i = 1/n$ para todo i). A matriz M é construída a partir da matriz de adjacência: $M_{ij} = 1/\text{out}(j)$ se há uma aresta de j para i , e 0 caso contrário. Para vértices sem arestas de saída (*dead ends*), são feitos ajustes para garantir a estocasticidade (Secção 3.3.2).

Trata-se de um **processo iterativo**: a equação $\mathbf{r}^{(t+1)} = d M \mathbf{r}^{(t)} + (1 - d) \mathbf{e}$ é aplicada repetidamente a partir de uma estimativa inicial $\mathbf{r}^{(0)}$. Sob as condições garantidas pelo amortecimento (que torna a cadeia de Markov ergódica), a sequência $\{\mathbf{r}^{(t)}\}_t$ converge para um único vetor estacionário $\mathbf{r}^*$ (Brin & Page, 1998; Masuda et al., 2017). Na prática, a iteração é interrompida quando $\|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}\|$ desce abaixo de uma tolerância pré-definida.

Ilustração. Considere o grafo direcionado simples com vértices A, B, C e arestas $A \rightarrow B, A \rightarrow C, B \rightarrow C, C \rightarrow A$. Os graus de saída são $A : 2, B : 1, C : 1$, donde $M = \begin{pmatrix} 0 & 0 & 1 \\ 0,5 & 0 & 0 \\ 0,5 & 1 & 0 \end{pmatrix}$ (colunas somam 1). Aplicando o PageRank com $d = 0,85$ e $\mathbf{e}$ uniforme, obtém-se o vetor de *ranks* após convergência (Figura 2.3).

Exemplo de grafo para cálculo do PageRank

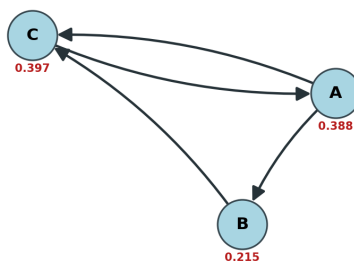


Figura 2.3 – Grafo exemplo usado para ilustrar o PageRank.

2.2.3 HITS (Hyperlink-Induced Topic Search)

O **HITS**, proposto por Kleinberg (1999), distingue dois papéis para os vértices: **autoridades** (páginas com conteúdo relevante) e **hubs** (páginas que apontam para boas autoridades). A ideia é que uma boa

autoridade é apontada por muitos bons *hubs*, e um bom *hub* aponta para muitas boas autoridades. Esta relação de reforço mútuo é expressa, de forma iterativa, por:

$$\mathbf{a} = A^T \mathbf{h}, \quad \mathbf{h} = A \mathbf{a} \quad (2.3)$$

onde A é a matriz de adjacência (para grafos direcionados). Após normalização, os vetores de autoridade $\mathbf{a}$ e *hub* $\mathbf{h}$ convergem para os autovetores principais de $A^T A$ e AA^T , respectivamente (Kleinberg, 1999; Ng et al., 2001). A Figura 2.4 ilustra a aplicação ao mesmo grafo exemplo.

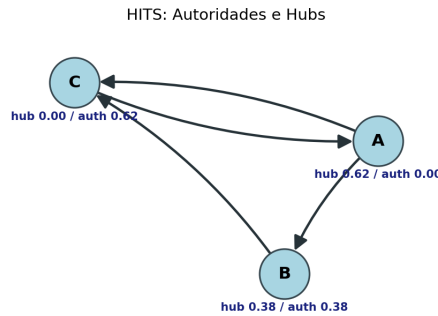


Figura 2.4 – Grafo exemplo usado para ilustrar o HITS.

2.2.4 SALSA (Stochastic Approach for Link-Structure Analysis)

O SALSA, proposto por Lempel e Moran (2001), combina ideias do PageRank e do HITS num enquadramento estocástico. A sua intuição é a seguinte: em vez de aplicar HITS diretamente sobre o grafo direcionado original, o SALSA constrói **dois passeios aleatórios em um grafo bipartido auxiliar** em que cada vértice do grafo original é representado duas vezes — uma como *hub* e outra como *authority*. Os dois passeios alternam entre estes dois lados:

- O passeio das **autoridades** parte de uma autoridade v , segue uma aresta no sentido inverso (até um *hub* u que a aponte) e depois segue uma aresta para a frente (até outra autoridade w apontada por u). A distribuição estacionária deste passeio define o *score* de autoridade de cada vértice.
- O passeio dos **hubs** é simétrico: parte de um *hub*, segue uma aresta para a frente até uma autoridade e regressa, no sentido inverso, a outro *hub*.

Sob normalização local (probabilidades $1/\text{out}(u)$ e $1/\text{in}(v)$), em **grafos não ponderados e fortemente conexos** a distribuição estacionária do passeio das autoridades reduz-se a $\text{in}(v)/|E|$, ou seja, é diretamente proporcional ao grau de entrada do vértice (Lempel & Moran, 2001). De forma análoga, o *score* de *hub* é proporcional ao grau de saída. Esta interpretação simples torna o SALSA computacionalmente mais leve que o HITS, mas tem uma consequência empírica relevante para esta dissertação: em grafos pequenos com graus pouco diferenciados, o SALSA pode produzir *distribuições praticamente uniformes*, o que se traduz num *ranking achatado* (ver Capítulo 4, Secção 4.5).

A vantagem teórica do SALSA face ao HITS é a sua resistência ao **Efeito TKC** (*Tightly Knit Community Effect*). Em grafos onde existem **comunidades muito coesas** — pequenos subconjuntos

SALSA: Autoridade (grau de entrada) e Hub (grau de saída)

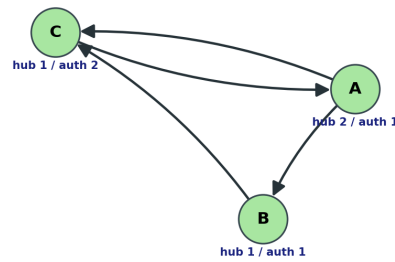


Figura 2.5 – Grafo exemplo usado para ilustrar o SALSA: os *ranks* são proporcionais aos graus de entrada (*authority*) e de saída (*hub*).

de vértices densamente interligados — o HITS tende a concentrar quase toda a massa dos *scores* nesses subgrupos, ainda que sejam pouco representativos do grafo global. Isto acontece porque o cálculo de autovetor reforça quem está fortemente interligado, mesmo que isoladamente. O SALSA, ao normalizar localmente por grau em cada passo do passeio, distribui a massa de forma mais equilibrada e evita esta concentração espúria (Lempel & Moran, 2001).

Caveat metodológico

A vantagem do SALSA face ao Efeito TKC só se observa empiricamente quando o corpus contém grafos com sub-comunidades densas embutidas em estruturas maiores. Os três grafos analisados nesta dissertação são demasiado pequenos para apresentarem tal fenómeno; consequentemente, a propriedade só é confirmada no plano teórico e **não** é empiricamente observada (ver Capítulo 4, Secção 4.5).

2.2.5 BiRank e normalizações simétricas

O **BiRank**, proposto por He et al. (2014), é um algoritmo projetado para grafos bipartidos ponderados. A diferença em relação ao PageRank reside na forma de **normalizar a matriz de pesos** antes da iteração:

- A **normalização estocástica** (usada pelo PageRank e por HITS) divide cada coluna j da matriz W pelo somatório dos pesos de saída do vértice j — ou seja, divide por $\sum_i W_{ij}$. Esta normalização torna as colunas estocásticas (somam 1) e dá uma interpretação probabilística direta a cada transição.
- A **normalização simétrica** divide cada entrada W_{ij} por $\sqrt{d_i \cdot d_j}$, onde d_i e d_j são os graus (ponderados) dos vértices envolvidos. O efeito desta segunda forma é **atenuar a influência desproporcional de vértices de grau muito elevado** (que dominariam, sob normalização estocástica), produzindo *scores* mais estáveis em grafos com distribuições de grau muito assimétricas (He et al., 2014; Zhou et al., 2004).

A iteração do BiRank é:

$$\mathbf{r}_{k+1} = \alpha S \mathbf{r}_k + (1 - \alpha) \mathbf{q} \quad (2.4)$$

onde $S = D_u^{-1/2} W D_v^{-1/2}$ é a matriz simetricamente normalizada, W é a matriz de pesos do grafo bipartido, D_u e D_v são as matrizes diagonais dos graus (em cada lado da bipartição) e $\mathbf{q}$ é um vetor de consulta (*prior information*). O BiRank é flexível para extensões a grafos multipartidos (He et al., 2015).

2.2.6 Comparação de rankings: correlação de Spearman e Kendall

Os algoritmos descritos nas secções anteriores produzem, cada um, uma **ordenação** dos vértices do grafo. Para avaliar em que medida dois algoritmos concordam — questão central desta dissertação (Secção 1.3) — não basta comparar os *scores* numéricos brutos, que vivem em escalas diferentes; compara-se antes a **ordem relativa** que cada algoritmo atribui aos vértices. Para esse efeito recorre-se a dois coeficientes clássicos de **correlação de ranks**, que constituem os *critérios formais de avaliação dos resultados* deste trabalho.

Coefficiente de Spearman (ρ). O coeficiente de Spearman (Spearman, 1904) é a correlação de Pearson calculada sobre os *ranks* (posições ordinais) em vez dos valores originais. Mede a força de uma relação **monotónica** entre dois rankings: vale +1 quando as duas ordenações são idênticas, -1 quando são exatamente inversas e 0 quando não há relação ordinal. Na ausência de empates, admite a forma fechada

$$\rho = 1 - \frac{6 \sum_i d_i^2}{n(n^2 - 1)},$$

onde d_i é a diferença de posição do elemento i entre os dois rankings e n o número de elementos.

Coefficiente de Kendall (τ_b). O coeficiente de Kendall (Kendall, 1938) baseia-se na contagem de **pares concordantes e discordantes**: dois elementos formam um par concordante se a sua ordem relativa é a mesma nos dois rankings, e discordante caso contrário. A variante τ_b (Goodman & Kruskal, 1954) introduz uma correção para **empates** (*ties*) — frequentes em grafos pequenos, onde vários vértices recebem *scores* idênticos. Tende a ser mais conservador do que ρ e tem interpretação probabilística direta: aproxima a diferença entre a probabilidade de concordância e a de discordância de um par escolhido ao acaso.

Ambos os coeficientes tomam valores em $[-1, 1]$. A formalização operacional usada na experimentação — incluindo o tratamento de *ties* via *mean ranks* e o critério *a priori* de “convergência forte” ($\rho \geq 0,7$ e $\tau_b \geq 0,55$) — é detalhada na Secção 3.4; os valores obtidos para cada par de algoritmos em cada caso de estudo são reportados e discutidos na Secção 4.3.

2.3 Análise estática e Árvores de Sintaxe Abstrata (AST)

Os algoritmos das secções anteriores operam sobre um grafo que, neste trabalho, **não** é fornecido manualmente, mas **extraído automaticamente do código-fonte**. O mecanismo que torna essa extração possível é a **análise estática** via *Abstract Syntax Tree* (AST). Por fazer parte do fluxo principal do processo (Capítulo 3), o conceito é apresentado aqui.

2.3.1 O que é uma AST

A **análise estática** examina o código-fonte *sem o executar*, em contraste com a análise dinâmica, que observa o programa em tempo de execução. O seu ponto de partida é a **Árvore de Sintaxe Abstrata**: uma representação em árvore da estrutura sintática do programa, em que cada nó corresponde a uma construção da linguagem — uma definição de função, uma chamada, uma atribuição, um laço. Diz-se “abstrata” porque omite detalhes da escrita concreta (parênteses, espaços, vírgulas) e retém apenas a estrutura sintático-semântica relevante.

Em Python, a AST é produzida pelo módulo `ast` da biblioteca padrão: a função `ast.parse(codigo)` devolve a árvore, que pode ser percorrida para localizar nós de interesse. Para este trabalho, os nós relevantes são as **definições de função** (`FunctionDef`), os **métodos de classe** e, sobretudo, as **invocações** (`Call`) — é a partir destas que se infere “que função chama que função”.

Exemplo. Para o fragmento `def a(): return b()`, a AST contém um nó `FunctionDef` (nome `a`) cujo corpo inclui um nó `Call` ao identificador `b`. Percorrendo a árvore, deteta-se a aresta de chamada $a \rightarrow b$ sem nunca executar o código — e, portanto, de forma **determinística**: a mesma entrada produz sempre o mesmo grafo.

2.3.2 Da AST ao Grafo de Requisitos: validação por fase

A extração do Grafo de Requisitos a partir da AST não é um passo único, mas um *pipeline* de quatro fases — `AST` $\rightarrow$ `Call Graph` $\rightarrow$ mapeamento `func_to_req` $\rightarrow$ Grafo de Requisitos — descrito em detalhe na Secção 3.2. Coloca-se então uma questão metodológica legítima: *como se garante que o output de cada fase está correto e que o grafo final representa fielmente as dependências?* A resposta assenta em dois pilares.

Determinismo e verificabilidade por fase. Por se basear exclusivamente em análise estática, cada fase produz um artefacto *inspecionável e reproduzível*, o que permite validar a correção de forma **incremental** — e não apenas no produto final:

- A **AST** é gerada pelo *parser* oficial da linguagem (módulo `ast`); a sua correção é garantida pela própria *toolchain* do Python e não introduz variabilidade.
- O **Call Graph** é exportado como artefacto visual (`call_graph.png`) e pode ser confrontado com a estrutura de chamadas conhecida do código — arestas em falta ou espúrias tornam-se detetáveis por inspeção.
- O **Grafo de Requisitos** é exportado em formato legível (`req_graph.json`, `req_graph.dot`) e visual (`req_graph.png`); a coerência das suas arestas com a arquitetura conhecida do projeto é verificada manualmente no passo de *validação de coerência* do protocolo (Secção 3.5.2).

A **noção de “requisito” adotada**. É importante sublinhar que o conceito de requisito usado nesta dissertação **não** é o de Sommerville (2007) — um enunciado de necessidade do sistema —, mas sim

uma **agregação de responsabilidades implementadas**, cujas interdependências são derivadas objetivamente da análise estática do código (cf. nota ontológica na Secção 1.2). É precisamente esta opção que torna a validação tratável: as dependências entre requisitos não são juízos subjetivos a verificar contra a intenção dos *stakeholders*, mas **factos estruturais do código** — se o requisito r_a contém uma função que invoca uma função do requisito r_b , a dependência $r_a \rightarrow r_b$ existe *por construção*. A correção do grafo reduz-se, assim, à correção de duas coisas verificáveis: (i) a deteção de chamadas pela AST (determinística) e (ii) o mapeamento `func_to_req` — o único passo com subjetividade residual, cujo protocolo e limitações são tratados nas Secções 3.5.2 e 4.6.

Validação do grafo — síntese

A validade do Grafo de Requisitos é *construída fase a fase*, não aferida apenas no fim: a AST é correta por garantia da *toolchain*; o *Call Graph* e o Grafo de Requisitos são artefactos inspecionáveis; e a semântica das arestas decorre objetivamente da estrutura do código. O único passo não verificável automaticamente — o mapeamento `func_to_req` — é isolado, com protocolo explícito (Secção 3.5.2) e registado como ameaça à validade (Secção 4.6).

2.4 Aplicação das Métricas ao Ranking e Priorização de Requisitos

2.4.1 Fundamentos da priorização de requisitos e desafios

A Engenharia de Requisitos (ER) é um domínio crucial da Engenharia de Software, responsável pela definição e manutenção das necessidades do sistema (Sommerville, 2007). Em projetos de software complexos, a priorização de requisitos é uma atividade essencial para o planeamento eficaz (Berander & Andrews, 2005; Karlsson, 2002).

A atividade de rastreabilidade de requisitos estabelece as relações de dependência entre os requisitos, sendo essa a base estrutural para a priorização (Gotel & Finkelstein, 1994; Sommerville, 2007). A literatura identifica diversas técnicas tradicionais, como a **Comparação em Pares** (*Pair-wise Comparison*), que exige que os *stakeholders* comparem cada par de requisitos, sendo uma tarefa demorada e de alto esforço (Karlsson & Ryan, 1997; Karlsson et al., 2007). Outra técnica é a **Votação Acumulativa** (*100-Points Method*), que distribui pontos fixos entre *stakeholders*, sendo a prioridade final a soma dos pontos recebidos (Ahl, 2005; Leffingwell & Widrig, 2003). Técnicas como o **PERT/CPM** utilizam grafos para representar a sequência lógica de planeamento, permitindo uma ordenação topológica (Kerzner, 2009). O **AHP** (*Analytic Hierarchy Process*) (Saaty, 1980) é, historicamente, o método mais citado e usado em priorização de requisitos (Berander & Andrews, 2005; Karlsson & Ryan, 1997): estabelece uma hierarquia de critérios e calcula pesos por comparação aos pares com verificação de consistência, mas tem complexidade $O(n^2)$ que o torna inviável para conjuntos com mais de algumas dezenas de requisitos.

Entretanto, métodos tradicionais de Priorização de Requisitos de Software (PRS) sofrem com problemas intrínsecos de subjetividade e esforço (Achimugu et al., 2014; Bukhsh et al., 2020; Firesmith,

2004). Fatores como a in experiência dos *stakeholders*, a divergência na interpretação das escalas de prioridade e o foco excessivo em apenas um ponto de vista resultam em prioridades inconsistentes e “posições inválidas” (Silva et al., 2018). Estudos práticos demonstram que a Votação Acumulativa, por depender excessivamente da ação e da decisão do *stakeholder*, pode gerar um alto número de posições inválidas e não resolve a duplicidade envolvida em requisitos interdependentes (Silva et al., 2018). *Surveys* recentes (Achimugu et al., 2014; Bukhsh et al., 2020) confirmam estes desafios e identificam o *ranqueamento baseado em grafos* como uma das direções mais promissoras para superar as limitações das técnicas manuais.

2.4.2 Construção do grafo de requisitos e posicionamento face a Silva et al. (2018)

A aplicação de algoritmos de ranqueamento estrutural na PRS baseia-se na construção de um **grafo de requisitos** (Silva et al., 2018). Neste modelo, cada requisito é um vértice e as relações de dependência (obtidas pela rastreabilidade) são arestas direcionadas. Especificamente, se o Requisito *A* depende de *B*, um *link* de avanço origina-se em *A* e aponta para *B*.

O presente trabalho posiciona-se explicitamente como uma **evolução metodológica** do trabalho seminal de Silva et al. (2018), que propõe a aplicação do PageRank à priorização de requisitos. No estudo original, os autores: (i) restringem a análise ao **PageRank** isoladamente, sem comparação sistemática com outros algoritmos de ranqueamento; (ii) constroem o grafo de requisitos a partir de **matrizes de rastreabilidade preexistentes**, mantidas manualmente; e (iii) validam a abordagem com uma avaliação qualitativa junto de profissionais. A presente dissertação preserva o quadro conceitual de Silva et al. (2018) — grafo de requisitos como suporte da priorização — e estende-o em três dimensões:

1. **Alarga o leque de algoritmos**, comparando PageRank com HITS (Kleinberg, 1999) e SALSA (Lempel & Moran, 2001) sobre os mesmos grafos, recorrendo a métricas formais de *rank correlation* (Spearman ρ , Kendall τ_b) para quantificar a concordância.
2. **Automatiza a construção do grafo de requisitos** a partir do código-fonte via análise estática (AST), eliminando a dependência de uma matriz de rastreabilidade mantida manualmente. Este *trade-off* merece registo explícito: substitui-se uma fonte fiável mas dispendiosa (rastreabilidade validada por engenheiros) por outra mais escalável mas com validade de construto a discutir (Secção 4.6, ameaças à validade).
3. **Valida o pipeline sobre três projetos reais de código aberto** com perfis arquiteturais distintos, permitindo observar o comportamento dos algoritmos sob diferentes regimes de acoplamento.

2.4.3 Mapeamento dos algoritmos de ranqueamento para priorização

Este referencial teórico fundamenta a aplicação de algoritmos de ranqueamento de *links* — notavelmente PageRank, HITS e SALSA — para objetivar e reduzir o esforço na PRS (Ding et al., 2002; Silva et al., 2018). Esta abordagem transforma a rede de dependências de requisitos num grafo estrutural,

permitindo que a priorização seja determinada pela topologia da rede, e não predominantemente pela percepção subjetiva do *stakeholder*.

O PageRank, em particular, tem-se mostrado sólido para a priorização (Silva et al., 2018). Foi usado para medir a complexidade de relacionamentos em sistemas (Li & Yi, 2009) e para analisar o impacto de preocupações em requisitos (Jin et al., 2009).

2.4.4 Benefícios, limitações e considerações práticas

A priorização via PageRank tem-se mostrado vantajosa em comparação com métodos manuais como a Votação Acumulativa e o PERT/CPM, oferecendo os seguintes benefícios (He et al., 2014; Silva et al., 2018):

1. **Redução do esforço sem eliminação da subjetividade.** O PageRank diminui o envolvimento direto dos *stakeholders*, focando-se na estrutura de dependência, e é escalável para lidar com um alto número de requisitos. Importa, contudo, sublinhar que a subjetividade é *deslocada* (do julgamento direto para a construção do grafo), não eliminada — ponto desenvolvido em Seção 4.6.
2. **Solução de Interdependência:** O PageRank, por meio das suas iterações, consegue solucionar a duplicidade envolvida em requisitos interdependentes, uma limitação não resolvida pela Votação Acumulativa ou PERT/CPM.
3. **Ajuste Flexível de Prioridade:** É possível introduzir informações externas (prioridade técnica ou de negócio) no grafo por meio de **vértices artificiais** (Silva et al., 2018). Essa técnica opcional permite que o *rank* de um requisito e o seu fecho transitivo de dependências sejam ajustados.

Como consideração prática, embora o PageRank ainda possa resultar em posições inválidas, esta abordagem estrutural revelou-se mais favorável do que as técnicas manuais, apresentando menos inconsistências e maior consistência com a lógica de dependência do sistema (Silva et al., 2018). As limitações da abordagem incluem as vulnerabilidades topológicas, como o **Efeito TKC**, que o SALSA foi especificamente desenvolvido para mitigar (Lempel & Moran, 2001).

Capítulo 3

Metodologia

A presente investigação adota uma abordagem **quantitativa e experimental**, fundamentada na modelagem matemática de sistemas de software através da Teoria dos Grafos e na execução repetível do *pipeline* analítico sobre projetos reais de código aberto. O Capítulo 3 detalha: Secção 3.1 — a estratégia de investigação; Secção 3.2 — a arquitetura do protótipo *ReqGraph*; Secção 3.3 — as escolhas de implementação; Secção 3.5 — o corpus experimental, o protocolo `func_to_req` e os critérios de avaliação. A Secção 3.4 formaliza as métricas de comparação de rankings.

Conceitos-chave: abordagem quantitativa; análise estática; AST; *pipeline* reproduzível; protocolo experimental; *rank correlation*.

3.1 Estratégia de Investigação

A investigação está estruturada em quatro etapas. Em primeiro lugar, **fundamentou-se teoricamente** a aplicação de algoritmos de ranqueamento de grafos à Priorização de Requisitos de Software (PRS), conforme exposto no Capítulo 2, consolidando o entendimento sobre as diferenças estruturais entre o HITS (dualidade *hubs*/autoridades), o SALSA (passeios aleatórios em grafos bipartidos) e o PageRank (distribuição estacionária em cadeias de Markov). Em segundo lugar, **projetou-se e implementou-se** um protótipo — o *ReqGraph* — capaz de extrair automaticamente o grafo de dependências entre requisitos a partir do código-fonte de uma aplicação Python, recorrendo a análise estática via *Abstract Syntax Tree* (AST). Em terceiro lugar, **definiu-se um protocolo experimental** para aplicar os algoritmos PageRank, HITS e SALSA aos grafos extraídos de três projetos de código aberto de complexidade crescente. Em quarto lugar, **quantificou-se a concordância** entre rankings via Spearman ρ e Kendall τ_b , respondendo a RQ1, RQ2 e RQ3 (Secção 1.3).

O ambiente de experimentação é **local**, em Python 3.8+, recorrendo às bibliotecas `ast` (biblioteca padrão), `NetworkX` (manipulação de grafos), `NumPy/SciPy` (computação matricial) e `Matplotlib` (visualização). Esta escolha justifica-se pela necessidade de manipular matrizes de adjacência e realizar cálculos iterativos de autovetores de forma escalável, em ambiente totalmente reproduzível e sem dependência de serviços externos.

A opção por **análise estática** (em detrimento de instrumentação dinâmica) deve-se à reprodutibilidade dos resultados: o grafo extraído depende exclusivamente do código-fonte, sem variabilidade introduzida por entradas de execução. Esta opção é coerente com a literatura sobre rastreabilidade de requisitos, que defende a derivação determinística do grafo de dependências a partir de artefactos

estáveis do sistema (Gotel & Finkelstein, 1994; Rasiman et al., 2022; Silva et al., 2018).

A análise dos resultados focar-se-á em três dimensões: (i) **Consistência Algorítmica** — comparação entre os rankings gerados pelos três algoritmos, recorrendo a coeficientes de correlação de Spearman e Kendall (RQ1); (ii) **Caracterização Estrutural** — densidade, presença de ciclos e identificação dos vértices centrais em cada Grafo de Requisitos (RQ2); (iii) **Coerência Arquitetural** — comparação qualitativa entre os requisitos ranqueados como mais importantes e a arquitetura conhecida de cada projeto (RQ3).

3.2 Arquitetura do Protótipo ReqGraph

O *ReqGraph* é o componente que materializa a transformação do código-fonte num **Grafo de Requisitos** $G_R = (V_R, E_R)$, em que cada vértice $v \in V_R$ representa um requisito de domínio e cada aresta direcionada $(r_i, r_j) \in E_R$ traduz uma dependência de implementação detetada entre os requisitos r_i e r_j . O *pipeline* analítico é composto por quatro estágios sequenciais:

1. **Análise estática via AST.** Cada ficheiro Python (`.py`) do projeto é parseado pelo módulo `ast` da biblioteca padrão. O resultado é uma árvore sintática a partir da qual são extraídas definições de funções (`FunctionDef`), métodos de classes e invocações (`Call`).
2. **Construção do Call Graph.** As invocações detetadas dão origem a um grafo direcionado $G_C = (V_C, E_C)$ em que cada vértice corresponde a uma função (ou método) e cada aresta (f_i, f_j) indica que f_i invoca f_j . O algoritmo resolve referências entre módulos (via `import`), referências a métodos da própria classe (`self.method()`) e qualifica funções pelo seu *namespace* para evitar colisões de nomes.
3. **Mapeamento `func_to_req`.** É fornecido pelo investigador um dicionário que associa cada função a um requisito de domínio (e.g. `parse_args` $\rightarrow$ `REQ_PARSING`). Este artefacto é o ponto de contacto entre o nível técnico (funções) e o nível semântico (requisitos). Para reduzir a fricção da sua produção, foi também desenvolvido um *prompt* estruturado (`reqgraph/llm_prompt_mapping.md`) que permite gerar o mapeamento de forma assistida via modelos de linguagem de grande escala (ChatGPT, Claude, Gemini). O protocolo de construção deste mapeamento é descrito em detalhe na Secção 3.5.2.
4. **Derivação do Grafo de Requisitos.** Para cada aresta $(f_i, f_j) \in E_C$ tal que f_i está mapeada para o requisito r_a e f_j está mapeada para o requisito r_b , com $r_a \neq r_b$, é introduzida a aresta (r_a, r_b) em E_R . Arestas duplicadas e auto-laços (correspondentes a invocações internas dentro do mesmo requisito) são descartados.

O código-fonte completo do *ReqGraph*, incluindo o módulo `ranker.py` descrito na Secção 3.3, está disponível em https://github.com/tales96323/Tales_Tese_ulusofona.

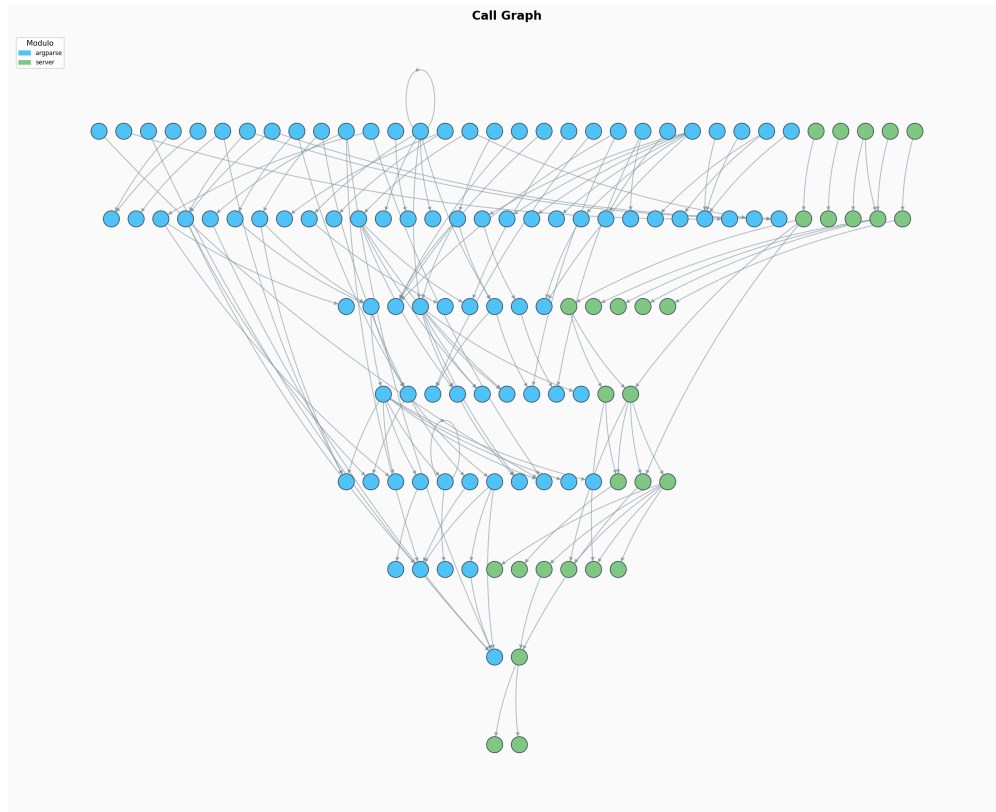


Figura 3.1 – *Call Graph* extraído via AST do CPython *stdlib* (`argparse` + `http.server`). Cada vértice é uma função ou método; cada aresta (f_i, f_j) indica que f_i invoca f_j .

3.3 Implementação e Tecnologias

O protótipo foi implementado em **Python 3.8+**, organizado como pacote instalável via `pip install -e reqgraph/`. As escolhas tecnológicas seguem critérios de robustez e adesão a *standards* da comunidade científica (Tabela 3.1).

Tabela 3.1 – Tecnologias utilizadas e respectivas funções no *pipeline*.

Tecnologia	Versão mín.	Função no <i>pipeline</i>
ast (biblioteca padrão)	—	<i>Parsing</i> sintático do código-fonte Python
networkx	≥ 3.0	Manipulação e análise de grafos direcionados
matplotlib	≥ 3.5	Geração de visualizações em PNG
numpy / scipy	≥ 1.24 / ≥ 1.10	Computação matricial
graphviz (DOT)	—	Formato de exportação canônico

A interface de linha de comando (`python -m reqgraph <projeto> --mapping <mapeamento.py>`) expõe o *pipeline* completo, gerando como artefactos: `call_graph.png`, `req_graph.png`, `req_graph.json` (lista de adjacência) e `req_graph.dot` (formato Graphviz).

3.3.1 Módulo de Ranqueamento (`ranker.py`)

O módulo `ranker.py` consome o ficheiro `req_graph.json` produzido pelo *ReqGraph* e aplica três algoritmos de ranqueamento sobre o grafo direcionado G_R . A semântica adotada para as arestas é: $r_a \rightarrow r_b$ significa que o requisito r_a **depende de** r_b . Em consequência, o *in-degree* de um vértice mede quantos requisitos dele dependem (uma autoridade no sentido de Kleinberg), enquanto o *out-degree* mede quantos outros requisitos ele invoca (um *hub*).

3.3.2 PageRank

A implementação recorre a `nx.pagerank` com parâmetros canónicos: fator de amortecimento $d = 0,85$, número máximo de iterações 100 e tolerância de convergência 10^{-6} .

Tratamento de *Dangling Nodes*. Vértices sem arestas de saída são tratados internamente pelo NetworkX por **redistribuição uniforme**: a massa de probabilidade que um vértice sem arestas de saída teria de “exportar” é dividida em partes iguais entre todos os vértices do grafo, mantendo a matriz de transição estocástica. Este artifício é necessário para garantir que a cadeia de Markov subjacente é ergódica e, portanto, que a iteração converge para uma única distribuição estacionária (Brin & Page, 1998). A soma dos *scores* totaliza 1,0, permitindo interpretá-los como uma distribuição de probabilidade.

3.3.3 HITS

A implementação recorre a `nx.hits` com tolerância 10^{-8} e 100 iterações máximas. Retorna dois vetores normalizados (norma euclidiana unitária): *hubs* e *authorities*. A interpretação para a PRS é direta: um requisito com elevado *authority* é uma **dependência crítica** do sistema (alterações nele propagam-se a muitos consumidores), enquanto um requisito com elevado *hub* é um **orquestrador** (alterações nele afetam muitas integrações).

3.3.4 SALSA

O NetworkX **não fornece** uma implementação do SALSA, pelo que esta foi desenvolvida manualmente seguindo Lempel e Moran (2001). Para cada aresta $(u, v) \in E_R$, constroem-se duas matrizes de transição:

- $H \rightarrow A$: probabilidade $1/\text{out}(u)$ de um *hub* u transitar para a *authority* v .
- $A \rightarrow H$: probabilidade $1/\text{in}(v)$ de uma *authority* v regressar a um *hub* u .

A cadeia de *hubs* tem matriz $T_{\text{hub}} = (A \rightarrow H)(H \rightarrow A)$, e a de *authorities* tem $T_{\text{auth}} = (H \rightarrow A)(A \rightarrow H)$. Os *scores* finais são as distribuições estacionárias obtidas por *power iteration* com tolerância 10^{-8} . Linhas nulas (correspondentes a vértices absorventes) são tratadas por redistribuição uniforme, em analogia ao tratamento de *dangling nodes* do PageRank.

A diferença substantiva face ao HITS é a **normalização local** (por grau de cada vértice), que torna o SALSA teoricamente menos sensível ao **Efeito TKC**. Como discutido em Secção 2.2.4, esta vantagem só é observável empiricamente em grafos com sub-comunidades coesas embutidas; o corpus utilizado nesta dissertação não permite, por dimensão, observar este fenómeno.

3.3.5 Artefactos Gerados

Para cada projeto analisado, o `ranker.py` produz dois artefactos: `ranking_results.json` (com os *scores* completos dos três algoritmos) e `ranking_results.png` (com gráficos de barras horizontais agrupados por algoritmo). Quando executado com a *flag* `--all`, gera adicionalmente `ranking_consolidado.json` e `ranking_comparativo.png` na raiz do projeto, agregando os resultados dos três casos de estudo. A partir de R5, é também produzido `rank_correlations.json` pelo *script* auxiliar `compute_rank_correlations.py`, que contém os coeficientes Spearman e Kendall para cada par de algoritmos em cada caso de estudo.

3.4 Métricas de Comparação de Rankings

A comparação entre rankings produzidos por algoritmos diferentes — elemento central de RQ1 e RQ2 — é feita de forma formal, recorrendo a duas métricas clássicas de *rank correlation*:

Coefficiente de Spearman (ρ). Pearson aplicado aos *ranks* (em vez dos valores brutos), com tratamento de *ties* via *mean ranks*. Mede correlação monotónica (Spearman, 1904).

$$\rho = 1 - \frac{6 \sum_i d_i^2}{n(n^2 - 1)} \quad (\text{sem ties})$$

Com *ties*, é calculado como a correlação de Pearson dos *mean ranks* (definição usada no script `compute_rank_correlations.py`).

Coefficiente de Kendall (τ_b). Baseado em pares concordantes e discordantes (Kendall, 1938); τ_b é a variante com correção de *ties* (Goodman & Kruskal, 1954):

$$\tau_b = \frac{|C| - |D|}{\sqrt{(P_0 - T_a)(P_0 - T_b)}}$$

onde $|C|$ e $|D|$ são os números de pares concordantes e discordantes, $P_0 = n(n-1)/2$ e T_a, T_b contam pares com *ties* em cada ranking.

Crítério a priori de “convergência”. Para fins de discussão (Secção 4.5), define-se *a priori*, antes de observar os dados, que existe **convergência entre dois algoritmos** quando se verifica qualquer um dos seguintes critérios:

- (a) Spearman $\rho \geq 0,7$ e Kendall $\tau_b \geq 0,55$ (ambos os limiares correspondem ao limiar usual de “*strong correlation*” na literatura (Wohlin et al., 2012));

(b) *Top-1* idêntico em pelo menos dois dos três algoritmos.

Quando nenhum dos critérios se verifica, fala-se de **divergência**; nesse caso, a interpretação é qualitativa (cada algoritmo capta uma dimensão diferente de centralidade, conforme discussão em Secção 4.5).

3.5 Corpus Experimental e Protocolo de Avaliação

3.5.1 Seleção dos Casos de Estudo

Para validar a abordagem em diferentes regimes de complexidade arquitetural, foram selecionados **três projetos reais de código aberto** disponíveis no GitHub, organizados por nível crescente de acoplamento esperado (Tabela 3.2). A escolha procura cobrir *regimes distintos*: biblioteca padrão monolítica (CPython *stdlib*), *microframework* web (Flask) e *framework* de *machine learning* com forte composição funcional (*scikit-learn*). A seleção privilegia ficheiros representativos do *core* arquitetural de cada projeto, evitando módulos auxiliares (testes, utilitários, *type stubs*).

Tabela 3.2 – Projetos selecionados como casos de estudo.

Nível	Projeto	Ficheiros Analisados	Origem (GitHub)
Simples	CPython <i>stdlib</i>	Lib/argparse.py, Lib/http/server.py	https://github.com/python/cpython
Médio	Flask	src/flask/app.py, src/flask/cli.py, src/flask/blueprints.py	https://github.com/pallets/flask
Complexo	<i>scikit-learn</i>	sklearn/pipeline.py, sklearn/linear_model/_base.py, sklearn/linear_model/_logistic.py, sklearn/tree/_classes.py	https://github.com/scikit-learn/scikit-learn

3.5.2 Protocolo de Construção do Mapeamento `func_to_req`

Dado que o passo (3) do *pipeline* (Secção 3.2) é a principal fonte de subjetividade residual, o presente trabalho formaliza explicitamente um protocolo para a sua construção:

1. **Levantamento de domínios candidatos.** Identificar, a partir da documentação oficial do projeto, módulos principais e *API* pública, um conjunto inicial de *features* de alto nível (e.g. *routing*, *request handling*, *error handling* no Flask).
2. **Definição de granularidade.** Cada requisito deve agregar entre 5 e 30 funções; conjuntos com menos de 5 funções fundem-se com domínios próximos; conjuntos com mais de 30 dividem-se segundo a sub-estrutura modular.
3. **Atribuição.** Cada função do código-fonte é atribuída a exatamente um requisito; funções utilitárias partilhadas por vários requisitos são atribuídas ao requisito do qual são chamadas mais frequentemente.

4. **Revisão de cobertura.** Verificar que todas as funções identificadas pela AST estão mapeadas; funções não mapeadas tornam-se *dangling nodes* no *Call Graph* e influenciam o ranking via redistribuição uniforme.
5. **Validação de coerência.** Inspeccionar manualmente o `req_graph.png` gerado: arestas inesperadas (e.g. ciclos que contradigam a arquitetura conhecida) podem indicar erros de mapeamento; arestas ausentes (e.g. ausência de dependências esperadas) podem indicar requisitos demasiado granulares.

Limitação reconhecida

A presente dissertação executa este protocolo com um único investigador, sem *inter-rater reliability* (e.g. Cohen's κ). A replicação por um segundo investigador, com cálculo formal da concordância, fica identificada como trabalho futuro (Secção 5.2) e como *ameaça de validade interna* (Secção 4.6).

3.5.3 Protocolo Experimental

Cada caso de estudo foi processado seguindo os mesmos seis passos, automatizados pelo *script* `run_tests.py`:

1. Cópia dos ficheiros-alvo do repositório de origem para a pasta `testes/<nivel>/`.
2. Construção manual do dicionário `func_to_req` em `mapeamento.py`, seguindo o protocolo da Secção 3.5.2.
3. Execução do *ReqGraph*:

```
python -m reqgraph testes/<nivel>/ --mapping testes/<nivel>/mapeamento.py
```
4. Inspeção dos artefactos gerados (`call_graph.png`, `req_graph.png`, `req_graph.json`).
5. Execução do `ranker.py` sobre o `req_graph.json` gerado.
6. Cálculo dos coeficientes de Spearman e Kendall entre cada par de algoritmos, via `compute_rank_correlations.py`.

3.5.4 Dimensões de Análise

A análise dos resultados (Capítulo 4) é estruturada em três dimensões:

- **Resultados Técnicos** — métricas estruturais do *pipeline*: número de funções analisadas, arestas no *Call Graph*, requisitos identificados, arestas no Grafo de Requisitos e densidade.
- **Resultados Analíticos** — interpretação dos rankings: identificação dos requisitos mais críticos, comparação entre os três algoritmos com Spearman e Kendall, e discussão à luz da arquitetura conhecida de cada projeto.

- **Ameaças à Validade** — categorização das limitações segundo o referencial de Wohlin et al. (2012): *construct, internal, external* e *conclusion*.

Capítulo 4

Avaliação de Resultados

O Capítulo 4 apresenta os resultados obtidos. A Secção 4.1 reúne as métricas técnicas dos três casos de estudo. A Secção 4.2 apresenta os rankings produzidos pelos três algoritmos. A Secção 4.3 quantifica formalmente a concordância entre algoritmos via Spearman ρ e Kendall τ_b (resposta a RQ1 e RQ2). A Secção 4.4 consolida a comparação entre projetos. A Secção 4.5 discute as observações principais (resposta a RQ3). A Secção 4.6 enuncia formalmente as ameaças à validade.

Conceitos-chave: resultados técnicos; resultados analíticos; correlação de ranks; convergência *a priori*; divergência informativa; ameaças à validade.

4.1 Resultados Técnicos

Esta secção sintetiza as métricas estruturais obtidas em cada caso de estudo, evidenciando a escalabilidade do *pipeline* e o grau de acoplamento detetado em cada projeto.

4.1.1 Síntese Quantitativa

A Tabela 4.1 resume as principais métricas obtidas pelo *pipeline ReqGraph* nos três casos de estudo. As contagens reportadas distinguem (i) o número total de requisitos *mapeados* no ficheiro `mapeamento.py` e (ii) o número de requisitos *efetivamente presentes* no `req_graph.json` (i.e. com pelo menos uma aresta incidente). A distinção é metodologicamente relevante: requisitos isolados (*singletons*) não contribuem para o ranking estrutural — são discutidos como *ameaça à validade interna* na Secção 4.6.

Discussão das contagens. Observa-se que o número de funções analisadas não cresce monotonicamente com a complexidade percebida (o `argparse` da *stdlib* tem mais funções do que o `app.py` do Flask), mas o número de arestas no Grafo de Requisitos cresce de forma quase monotónica — passando de 9 (CPython) para 11 (Flask) e 17 (*scikit-learn*). Esta métrica é, portanto, mais informativa sobre o acoplamento arquitetural do que a contagem bruta de funções.

Requisitos isolados. O caso **Flask** merece atenção: dos 13 requisitos mapeados, apenas 7 aparecem no Grafo de Requisitos. Os 6 requisitos isolados — `REQ_BLUEPRINTS`, `REQ_CLI_DISCOVERY`, `REQ_CLI_TYPES`, `REQ_STATIC_FILES`, `REQ_TEMPLATING` e `REQ_TESTING` — correspon-

Tabela 4.1 – Síntese quantitativa dos três casos de estudo. *Densidade* é calculada como $|E|/(n(n-1))$ sobre requisitos com arestas.

Métrica	CPython <i>stdlib</i>	Flask	<i>scikit-learn</i>
Ficheiros analisados	2	3	4
Funções/métodos detetados	298	125	210
Arestas no <i>Call Graph</i>	130	47	80
Requisitos mapeados (mapeamento.py)	10	13	9
Requisitos com arestas no Grafo de Requisitos	9	7	8
Requisitos isolados (<i>singletons</i>)	1	6	1
Arestas no Grafo de Requisitos	9	11	17
Densidade $ E /(n(n-1))$	0,125	0,262	0,304
Resultado da execução	Passou	Passou	Passou

dem a funções que, dentro dos três ficheiros analisados, não invocam funções de outros requisitos nem são invocadas por elas. Esta observação é *informativa*: revela que parte significativa da arquitetura do Flask referenciada no mapeamento está implementada *fora* dos ficheiros analisados (e.g., a *templating engine* reside em `jinja2`, externa ao corpus). Trata-se de uma limitação direta da estratégia de seleção de ficheiros (Secção 3.5) e é endereçada como ameaça à *external validity* (Secção 4.6).

Densidade. Os valores de densidade 0,125, 0,262 e 0,304 correspondem ao crescimento esperado de acoplamento arquitetural. Importa, contudo, ressaltar (cf. Newman 2010): em grafos com poucas dezenas de vértices, a densidade é uma métrica *volátil* — pequenas variações no mapeamento podem deslocá-la significativamente. Por essa razão, na Secção 4.5 discutem-se também as variações de *clustering coefficient* e de presença de ciclos como métricas complementares.

4.1.2 Caso Simples — CPython *stdlib*

Foram analisados os módulos `argparse.py` (*parsing* de argumentos) e `http/server.py` (servidor HTTP simples). O mapeamento associa 298 funções a 10 requisitos de domínio, dos quais 9 aparecem no Grafo de Requisitos. O grafo resultante apresenta apenas 9 arestas, sem ciclos, e pode ser sintetizado pelas seguintes adjacências:

```
{
  "REQ_ACTIONS": ["REQ_CONFIGURATION", "REQ_FORMATTING", "REQ_VALIDATION"],
  "REQ_ERROR_HANDLING": ["REQ_FORMATTING"],
  "REQ_FORMATTING": ["REQ_ERROR_HANDLING"],
  "REQ_HTTP_HANDLER": ["REQ_HTTP_CONTENT", "REQ_HTTP_LOGGING"],
  "REQ_PARSING": ["REQ_ERROR_HANDLING", "REQ_VALIDATION"]
}
```

Listing 4.1 – Adjacências do Grafo de Requisitos — CPython *stdlib*.

O grafo divide-se claramente em dois sub-sistemas independentes (*parsing*/argparse vs. HTTP), confirmando o baixo acoplamento esperado de módulos da biblioteca padrão. O único ciclo identificado é o par `REQ_FORMATTING` ↔ `REQ_ERROR_HANDLING`, refletindo a coexistência das funções

`format_help` e `error`, que se chamam mutuamente.

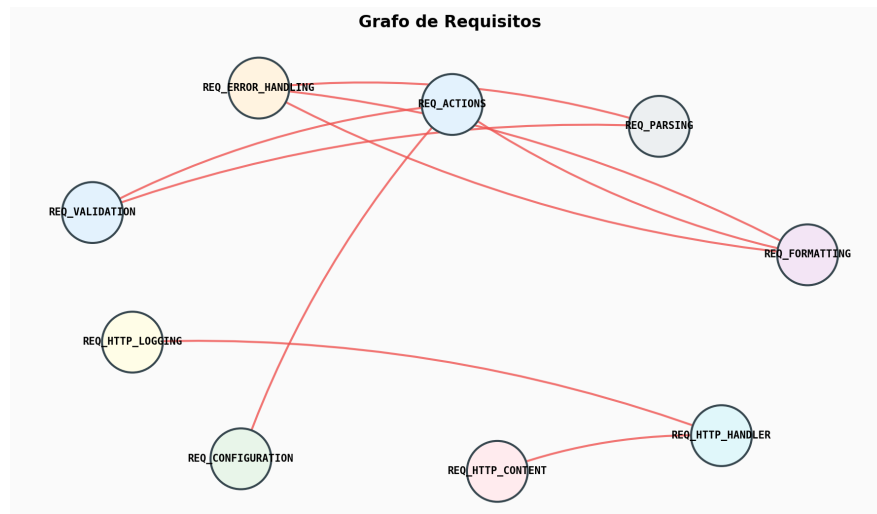


Figura 4.1 – Grafo de Requisitos: CPython *stdlib*. Note-se a divisão clara em dois sub-sistemas independentes (*parsing/argparse* vs. HTTP) e o ciclo `FORMATTING` ↔ `ERROR_HANDLING`.

4.1.3 Caso Intermediário — Flask

Foram analisados `app.py`, `cli.py` e `blueprints.py`, perfazendo 125 funções mapeadas em 13 requisitos, dos quais **apenas 7 aparecem no Grafo de Requisitos** (ver discussão em Secção 4.1.1). O Grafo de Requisitos contém 11 arestas e — diferentemente do caso anterior — apresenta **fortes relações cíclicas**:

```
{
  "REQ_APP_CORE": ["REQ_CLI_COMMANDS", "REQ_CLI_FRAMEWORK", "REQ_CONTEXT"],
  "REQ_CLI_FRAMEWORK": ["REQ_CONTEXT"],
  "REQ_CONTEXT": ["REQ_REQUEST_HANDLING"],
  "REQ_ERROR_HANDLING": ["REQ_REQUEST_HANDLING"],
  "REQ_REQUEST_HANDLING": ["REQ_CONTEXT", "REQ_ERROR_HANDLING", "REQ_ROUTING"],
  "REQ_ROUTING": ["REQ_ERROR_HANDLING", "REQ_REQUEST_HANDLING"]
}
```

Listing 4.2 – Adjacências do Grafo de Requisitos — Flask.

O ciclo `REQUEST_HANDLING` ↔ `CONTEXT` ↔ `ROUTING` ↔ `ERROR_HANDLING` evidencia que estas quatro responsabilidades estão estreitamente acopladas, o que é coerente com a arquitetura *middleware* do Flask, onde o ciclo de vida de uma requisição obriga à coordenação entre *dispatcher*, contexto de aplicação e tratamento de exceções.

Na Figura 4.2, as cores atribuídas aos vértices identificam o ficheiro-fonte do projeto a que cada requisito pertence (azul para `app.py`, verde para `cli.py`, laranja para `blueprints.py`), permitindo perceber visualmente como o acoplamento atravessa os módulos. As cores das arestas seguem a cor do vértice de origem, facilitando a leitura das cadeias de dependência.

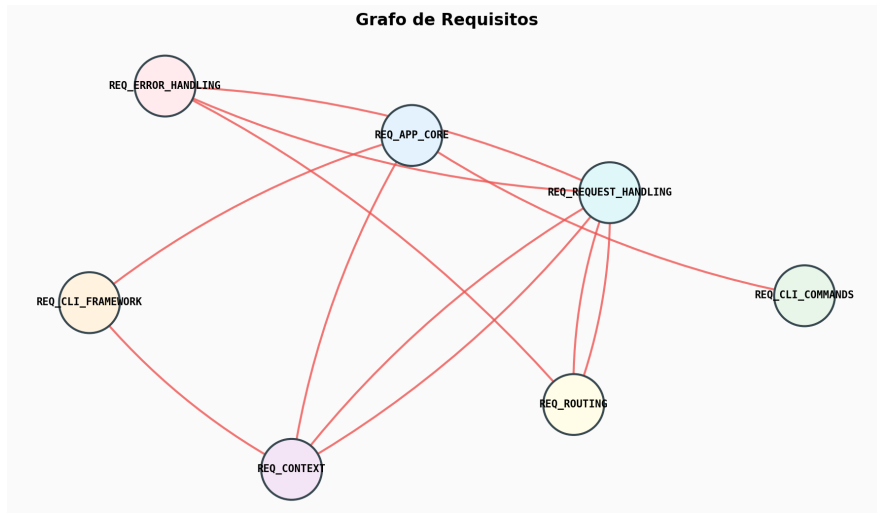


Figura 4.2 – Grafo de Requisitos: Flask. O ciclo `REQUEST_HANDLING ↔ CONTEXT ↔ ROUTING ↔ ERROR_HANDLING` evidencia o forte acoplamento típico do *middleware*.

4.1.4 Caso Complexo — *scikit-learn*

Foram analisados `pipeline.py`, `linear_model/_base.py`, `linear_model/_logistic.py` e `tree/_classes.py`, contabilizando 210 funções mapeadas em 9 requisitos, dos quais 8 aparecem no Grafo de Requisitos (`REQ_SPARSE` fica isolado). O Grafo de Requisitos resultante é o **mais denso** dos três casos (17 arestas, densidade 0,304), refletindo o elevado grau de orquestração interna do *framework*:

```
{
  "REQ_DECISION_TREE": ["REQ_SKLEARN_TAGS"],
  "REQ_FEATURE_UNION": ["REQ_PIPELINE"],
  "REQ_LOGISTIC_REGRESSION": ["REQ_PIPELINE_PREDICT"],
  "REQ_PIPELINE": ["REQ_LOGISTIC_REGRESSION", "REQ_PIPELINE_PREDICT", "
    REQ_SKLEARN_TAGS"],
  "REQ_PIPELINE_FIT": ["REQ_DECISION_TREE", "REQ_FEATURE_UNION", "REQ_LINEAR_MODEL",
    "REQ_LOGISTIC_REGRESSION", "REQ_PIPELINE"],
  "REQ_PIPELINE_PREDICT": ["REQ_DECISION_TREE", "REQ_FEATURE_UNION", "
    REQ_LINEAR_MODEL", "REQ_LOGISTIC_REGRESSION", "REQ_PIPELINE"],
  "REQ_SKLEARN_TAGS": ["REQ_PIPELINE"]
}
```

Listing 4.3 – Adjacências do Grafo de Requisitos — *scikit-learn*.

`REQ_PIPELINE_FIT` e `REQ_PIPELINE_PREDICT` conectam-se cada um a cinco outros requisitos, confirmando o seu papel de **orquestradores centrais**. `REQ_PIPELINE` recebe arestas de quatro requisitos distintos, sendo a **autoridade dominante** do sistema.

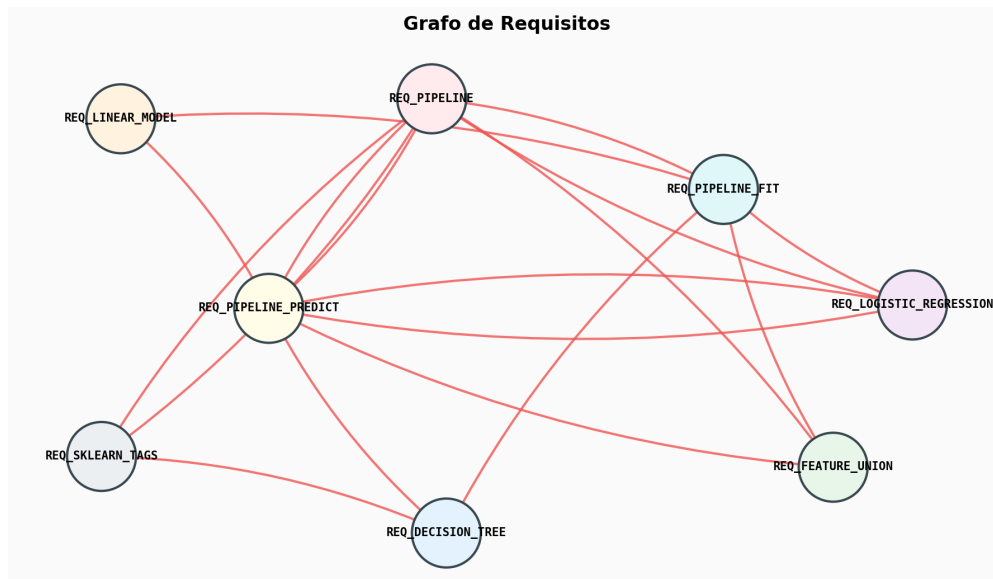


Figura 4.3 – Grafo de Requisitos: *scikit-learn*. O caso mais denso (17 arestas) reflete a orquestração interna do *framework*: REQ_PIPELINE_FIT e REQ_PIPELINE_PREDICT conectam-se cada um a cinco outros requisitos.

4.2 Rankings produzidos pelos três algoritmos

Esta secção apresenta os rankings produzidos pelo PageRank, HITS e SALSA para cada caso de estudo, interpretando os *scores* à luz da arquitetura conhecida dos projetos.

4.2.1 Caso Simples — Rankings para o CPython *stdlib*

A Tabela 4.2 reúne os três primeiros requisitos identificados por cada algoritmo. Os valores entre parênteses correspondem ao *score* numérico atribuído pelo respetivo algoritmo — para o PageRank, são probabilidades estacionárias (somam 1 no total do grafo); para o HITS, são componentes de vetores normalizados (norma euclidiana unitária); para o SALSA, são componentes de vetores normalizados por soma unitária (cada vetor — *authority* ou *hub* — soma 1). *Scores* mais elevados indicam maior centralidade segundo o critério do algoritmo.

Tabela 4.2 – Top-3 por algoritmo: CPython *stdlib*.

Algoritmo	1.º	2.º	3.º
PageRank	REQ_ERROR_HANDLING (0,337)	REQ_FORMATTING (0,334)	REQ_VALIDATION (0,064)
HITS — Authority	REQ_VALIDATION (0,338)	REQ_FORMATTING (0,280)	REQ_CONFIGURATION (0,209)
HITS — Hub	REQ_ACTIONS (0,462)	REQ_PARSING (0,285)	REQ_ERROR_HANDLING (0,156)
SALSA — Authority	REQ_HTTP_CONTENT (0,167)	REQ_HTTP_LOGGING (0,167)	REQ_ERROR_HANDLING (0,167)
SALSA — Hub	REQ_HTTP_HANDLER (0,200)	REQ_ERROR_HANDLING (0,200)	REQ_ACTIONS (0,200)

O PageRank identifica REQ_ERROR_HANDLING como o requisito mais crítico, concentrando 0,337 da probabilidade estacionária — ou seja, **um caminhante aleatório que percorra o grafo durante muito tempo passará cerca de 33,7% do tempo nesse vértice**. Esta é a interpretação direta da “massa de probabilidade” a que o PageRank dá origem: cada *score* indica que fração do tempo

um passeio aleatório de longa duração visita aquele vértice. REQ_FORMATTING segue muito de perto (0,334). Esta dupla decorre do ciclo FORMATTING $\leftrightarrow$ ERROR_HANDLING, que faz a probabilidade acumular-se nesses dois vértices via *random walk*. O HITS distingue claramente os papéis: REQ_ACTIONS e REQ_PARSING são *hubs* (consomem muitas dependências), enquanto REQ_VALIDATION, REQ_FORMATTING e REQ_CONFIGURATION são *authorities* (são consumidos por muitos). O SALSA, devido à sua normalização local, atribui *scores* idênticos a todos os vértices com arestas — comportamento esperado em grafos com graus pouco diferenciados (cf. Lempel e Moran 2001).

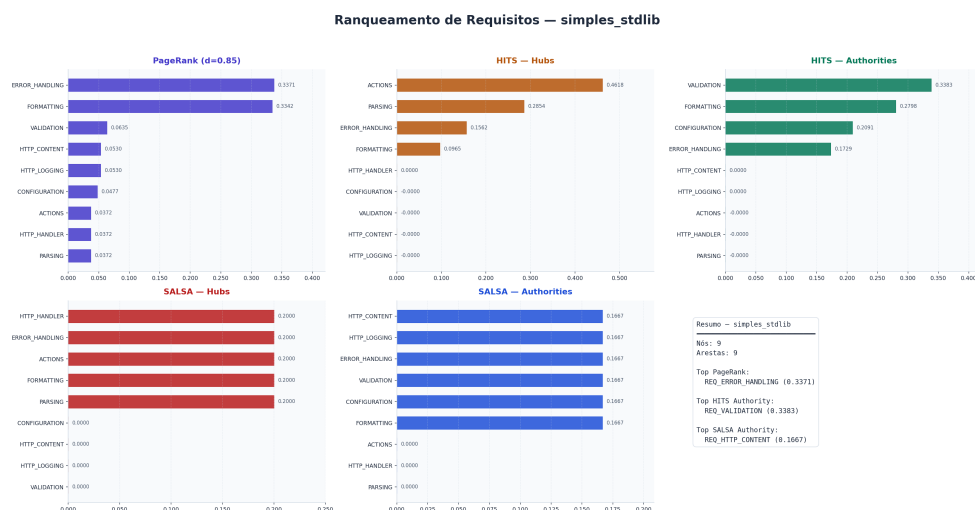


Figura 4.4 – Rankings produzidos pelos três algoritmos para o caso CPython *stdlib*.

4.2.2 Caso Intermediário — Rankings para o Flask

A Tabela 4.3 apresenta os *top-3* do caso médio. Para os algoritmos SALSA, “uniforme entre 6 requisitos” significa que seis vértices receberam o mesmo *score* ($1/6 \approx 0,167$); os requisitos isolados receberam *score* zero (não contam para o top).

Tabela 4.3 – *Top-3* por algoritmo: Flask.

Algoritmo	1.º	2.º	3.º
PageRank	REQ_REQUEST_HANDLING (0,400)	REQ_ERROR_HANDLING (0,198)	REQ_CONTEXT (0,174)
HITS — Authority	REQ_CONTEXT (0,318)	REQ_ERROR_HANDLING (0,204)	REQ_ROUTING (0,137)
HITS — Hub	REQ_REQUEST_HANDLING (0,319)	REQ_APP_CORE (0,264)	REQ_CLI_FRAMEWORK (0,154)
SALSA — Authority	<i>uniforme</i> a 0,167 entre 6 requisitos com arestas		
SALSA — Hub	<i>uniforme</i> a 0,167 entre 6 requisitos com arestas		

REQ_REQUEST_HANDLING emerge como o vértice central segundo o PageRank, com 40% da probabilidade estacionária — um valor anormalmente elevado que reflete a sua posição em três arestas de entrada e três de saída, todas dentro do ciclo principal. O HITS divide o protagonismo: REQ_REQUEST_HANDLING é simultaneamente *hub* e *authority* (acoplamento bidirecional típico do *middleware*), enquanto REQ_CONTEXT lidera as *authorities* puras. O SALSA produziu uma **distribuição praticamente uniforme**, alinhada com a propriedade demonstrada por Lempel e Moran (2001): em

grafos com graus pouco diferenciados, o *score* converge para uma constante por classe de equivalência de grau.

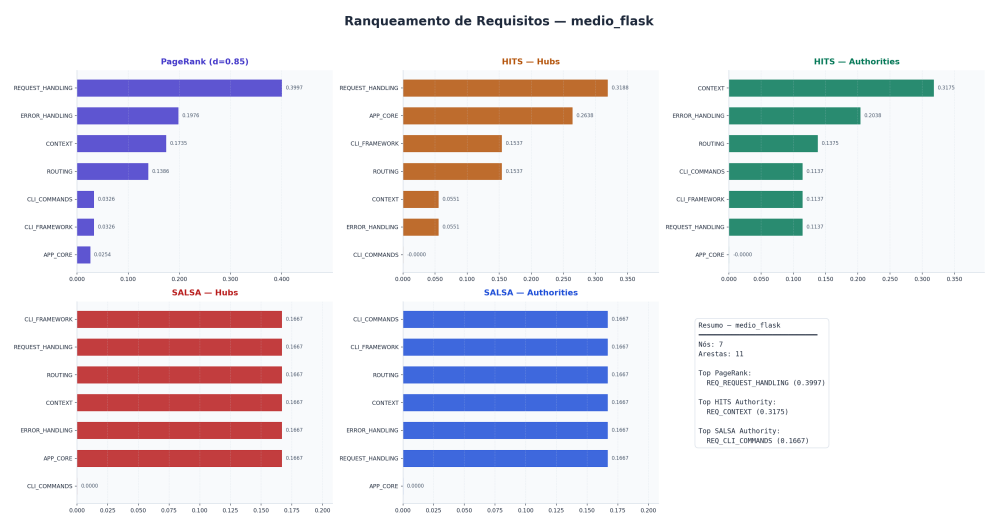


Figura 4.5 – Rankings produzidos pelos três algoritmos para o caso Flask.

4.2.3 Caso Complexo — Rankings para *scikit-learn*

A Tabela 4.4 apresenta o *top-3* do caso mais complexo.

Tabela 4.4 – Top-3 por algoritmo: *scikit-learn*.

Algoritmo	1.º	2.º	3.º
PageRank	REQ_PIPELINE (0,257)	REQ_PIPELINE_PREDICT (0,218)	REQ_SKLEARN_TAGS (0,18)
HITS — Authority	REQ_PIPELINE (0,217)	REQ_LOGISTIC_REGRESSION (0,200)	REQ_DECISION_TREE (0,18)
HITS — Hub	REQ_PIPELINE_PREDICT (0,360)	REQ_PIPELINE_FIT (0,360)	REQ_PIPELINE (0,096)
SALSA — Authority	uniforme a 0,143 entre 7 requisitos com arestas		
SALSA — Hub	uniforme a 0,143 entre 7 requisitos com arestas		

PageRank e HITS coincidem na identificação de REQ_PIPELINE como o requisito mais central: o PageRank atribui-lhe o *score* mais elevado (0,257) e o HITS classifica-o como a *authority* dominante (0,217). O HITS identifica adicionalmente REQ_PIPELINE_PREDICT e REQ_PIPELINE_FIT como co-líderes dos *hubs* com *scores* idênticos (0,360) — refletindo a sua simetria estrutural (ambos invocam exatamente os mesmos cinco requisitos). O SALSA volta a produzir uma distribuição uniforme entre os vértices com arestas, indicando que, neste corpus, não há vértices destacados pelas suas frações de grau.

4.3 Correlações Spearman e Kendall entre Rankings

Para responder formalmente a **RQ1** (em que medida os algoritmos concordam) e a **RQ2** (como a concordância varia com as propriedades estruturais), calcularam-se os coeficientes de correlação de Spearman (ρ) e Kendall (τ_b) entre cada par de algoritmos, em cada caso de estudo. Os valores foram

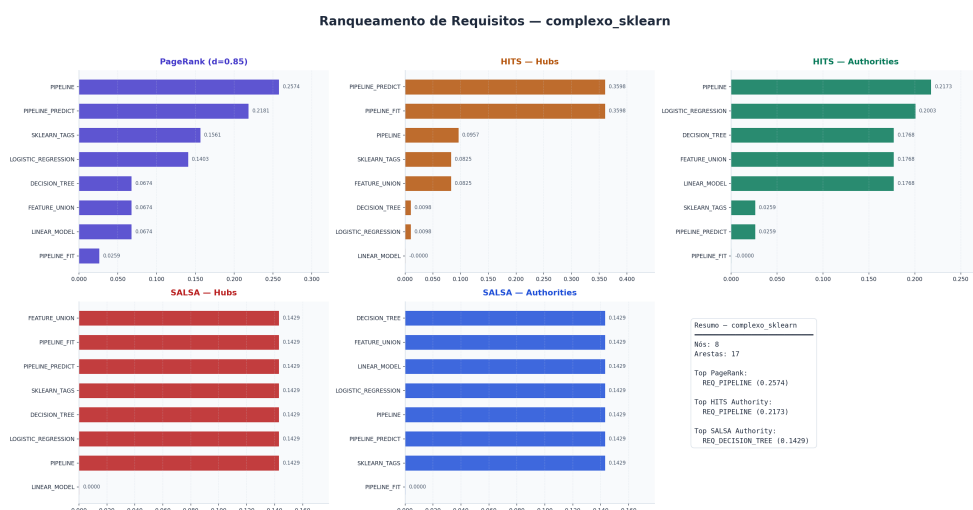


Figura 4.6 – Rankings produzidos pelos três algoritmos para o caso *scikit-learn*.

produzidos pelo *script* `compute_rank_correlations.py` e estão sintetizados na Tabela 4.5. Em `realçado` indica-se cada par que satisfaz o critério *a priori* de **convergência forte** (Secção 3.4: $\rho \geq 0,7$ e $\tau_b \geq 0,55$).

Tabela 4.5 – Coeficientes de correlação de ranks (Spearman ρ , Kendall τ_b) entre cada par de algoritmos, por caso de estudo. *Top-1* indica se o requisito *top-1* coincide.

Par	CPython <i>stdlib</i>			Flask			scikit-learn		
	ρ	τ_b	T1	ρ	τ_b	T1	ρ	τ_b	T1
PR vs HITS-auth	0,65	0,49	×	0,60	0,53	×	0,39	0,33	✓
PR vs HITS-hub	−0,15	−0,03	×	0,08	0,00	✓	0,24	0,24	×
PR vs SALSA-auth	0,84	0,75	×	0,62	0,55	×	0,59	0,53	×
PR vs SALSA-hub	−0,18	−0,16	×	0,31	0,27	×	0,25	0,23	×
HITS-auth vs SALSA-auth	0,60	0,55	×	0,64	0,58	×	0,60	0,54	×
HITS-hub vs SALSA-hub	0,76	0,70	×	0,62	0,56	×	0,59	0,53	×

Leitura por caso. No **CPython *stdlib***, dois pares satisfazem o critério de convergência forte: PR $\leftrightarrow$ SALSA-auth ($\rho = 0,84$, $\tau_b = 0,75$) e HITS-hub $\leftrightarrow$ SALSA-hub ($\rho = 0,76$, $\tau_b = 0,70$). Os *top-1*, contudo, são diferentes: o PageRank captura o ciclo ERROR_HANDLING $\leftrightarrow$ FORMATTING via massa estacionária, enquanto o SALSA reparte uniformemente.

No **Flask**, *nenhum par satisfaz o critério forte*; o único *top-1 match* ocorre entre PR e HITS-hub (REQ_REQUEST_HANDLING), num par que paradoxalmente tem $\rho = 0,08$ no ranking completo — indício de que o consenso sobre o vértice central não se estende ao restante *ranking*.

No **scikit-learn** — ostensivamente o caso “mais denso” e portanto, à partida, mais favorável à convergência — *nenhum par* satisfaz o critério forte. O único *top-1 match* ocorre entre PR e HITS-auth (REQ_PIPELINE). Este resultado mostra que **maior densidade não implica maior correlação global** entre algoritmos — contrariando a intuição inicial que motivava a tese (Capítulo 1).

Resposta a RQ1. A concordância entre algoritmos é **parcial e seletiva**: dos 18 pares (6 pares $\times$ 3 projetos), apenas dois satisfazem o critério forte de convergência; três adicionais aproximam-se ($\rho \geq 0,59$, $\tau_b \geq 0,53$). Os restantes treze divergem, alguns com correlação próxima de zero ou ligeiramente negativa.

Resposta a RQ2. Contrariamente à hipótese inicial, **a densidade do grafo não está positivamente correlacionada com a concordância entre algoritmos** no presente corpus. A *média* de ρ por projeto é $\bar{\rho}_{\text{stdlib}} = 0,43$, $\bar{\rho}_{\text{Flask}} = 0,48$ e $\bar{\rho}_{\text{sklearn}} = 0,44$ — valores praticamente indistinguíveis. O que parece variar com a densidade é o *padrão* de divergência, não a sua magnitude global. As diferenças observadas de densidade entre os três grafos (0,125, 0,262 e 0,304) não são, por si só, suficientemente acentuadas para sustentar generalizações fortes; a leitura aqui apresentada é uma caracterização dos três casos concretos.

4.4 Comparação Consolidada

A Figura 4.7 reúne, num único gráfico, os requisitos *top-1* identificados por cada algoritmo em cada um dos três casos de estudo.

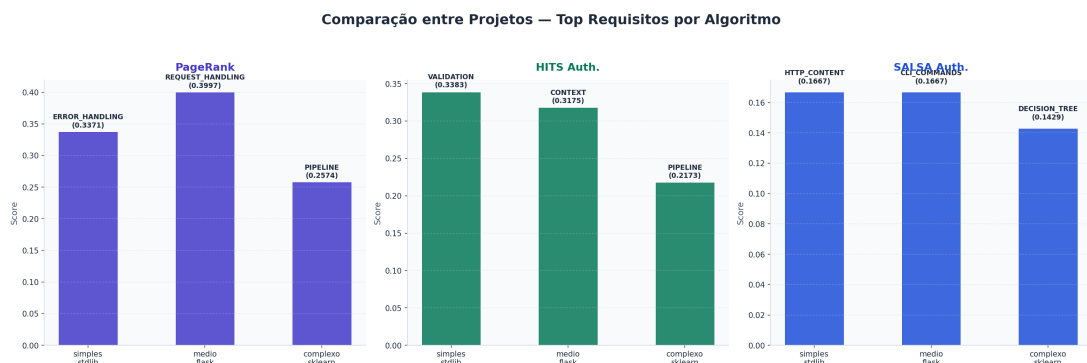


Figura 4.7 – Ranking comparativo entre os três projetos: *top-1* por algoritmo em cada caso de estudo. Compare-se com as Tabelas 4.2, 4.3 e 4.4.

4.5 Discussão

A análise cruzada dos resultados quantitativos (Secção 4.3) e qualitativos (Secção 4.2) permite extrair quatro observações principais:

1. **Concordância parcial, não unanimidade.** Apenas dois dos 18 pares (*stdlib*: PR vs SALSA-auth e HITS-hub vs SALSA-hub) satisfazem o critério *a priori* de convergência forte. Nos *top-1*, apenas *dois matches* ocorrem (Flask: PR/HITS-hub em REQ_REQUEST_HANDLING; *sklearn*: PR/HITS-auth em REQ_PIPELINE). A narrativa de “convergência global no caso mais denso” — presente em iterações anteriores deste trabalho — não se sustenta face aos coeficientes formais.

2. **Divergência informativa entre HITS e SALSA versus PageRank.** Os pares envolvendo *hubs* (PR vs HITS-hub, PR vs SALSA-hub) apresentam correlação muito baixa ou ligeiramente negativa em *stdlib* e Flask. Isto confirma que *hubs* e *authorities* medem dimensões distintas de centralidade — a *orquestração* é uma propriedade dual da *importância transitiva*. Esta divergência é **informativa**, não defeituosa: o engenheiro de requisitos pode usar PR para identificar “onde a importância flui”, HITS-auth para identificar “de quem todos dependem”, e HITS-hub para identificar “quem orquestra muitos”.
3. **Comportamento do SALSA em grafos pequenos — resultado nulo, não confirmação teórica.** O SALSA produziu *distribuições praticamente uniformes* em dois dos três casos (Flask e *scikit-learn*). Esta observação é consistente com a propriedade demonstrada por Lempel e Moran (2001) — em grafos não ponderados e fortemente conexos, o *score* de *authority* reduz-se a $\ln(v)/|E|$ —, mas **não constitui confirmação empírica da mitigação do Efeito TKC**. O Efeito TKC pressupõe a existência de *sub-comunidades densamente coesas* embutidas em estruturas maiores; nenhum dos três grafos analisados apresenta tal característica (cf. análise estrutural em Secção 4.1). Honestamente, este é um *resultado nulo*: o SALSA não distinguiu nada onde os outros algoritmos distinguiram — e a vantagem teórica do SALSA é *esperada* aparecer em corpora onde o TKC se manifesta, condição que o presente corpus não cumpre. A propriedade fica assim confirmada apenas no plano teórico (Capítulo 2).
4. **Acoplamento como propriedade emergente.** Os ciclos identificados no Flask e a alta densidade no *scikit-learn* não foram inseridos manualmente — emergem do código real. Isto sustenta a tese de que o **grafo de dependências contém informação latente** sobre a importância dos requisitos, suficiente para apoiar — mas não substituir — a decisão dos *stakeholders*.

Resposta a RQ3. A coerência arquitetural dos rankings é **moderada e direcionada**: os *top-1* identificados correspondem a elementos efetivamente centrais nas arquiteturas conhecidas (a *Pipeline* no *scikit-learn* é o coração do *framework*; o *request handling* no Flask é o *middleware* central). Contudo, esta correspondência é *qualitativa* — baseada na inspeção do investigador, não em juízo de profissionais externos. A validação humana fica como trabalho futuro (Secção 5.2).

Implicações práticas. Para um engenheiro de requisitos ou *project manager* confrontado com um sistema Python existente, os resultados sugerem um fluxo prático: (1) executar o *ReqGraph* para gerar o grafo de requisitos; (2) executar o `ranker.py` e inspecionar o *top-5* de cada um dos três algoritmos; (3) considerar *candidatos a requisito crítico* todos os requisitos que aparecem no *top-5* de pelo menos dois dos três algoritmos — a interseção tem maior probabilidade de captar centralidades genuínas e menos sensíveis a uma única definição de centralidade.

4.6 Ameaças à Validade

As ameaças à validade são apresentadas seguindo a taxonomia clássica de Wohlin et al. (2012), ordenadas por relevância dentro de cada categoria.

4.6.1 Validade de Construto (*Construct Validity*)

- **“Requisito” como agregação de funções é *proxy* discutível.** O artefacto `func_to_req` agrega funções em *domain features* que são interpretadas como requisitos. Esta operacionalização não coincide com a definição clássica de requisito funcional (Sommerville, 2007); corresponde antes a uma noção de *feature module*. O construto medido é, portanto, “centralidade estrutural de *feature module*”, e não “importância de requisito” no sentido pleno. Esta distinção foi explicitada na nota ontológica (Secção 1.2).
- **Subjetividade deslocada, não eliminada.** A priorização estrutural reduz o esforço dos *stakeholders*, mas desloca a subjetividade para (a) a escolha dos ficheiros analisados e (b) a construção do mapeamento `func_to_req`. A tese *não* demonstra que estes dois passos são menos subjetivos do que a Votação Acumulativa — apenas que ocorrem em momentos diferentes do processo (*up-front* vs. *per-requirement*).

4.6.2 Validade Interna (*Internal Validity*)

- **Validação do grafo concentrada num único passo subjetivo.** A correção do grafo é construída fase a fase (Secção 2.3.2): a AST é determinística e garantida pela *toolchain*; o *Call Graph* e o Grafo de Requisitos são artefactos inspecionáveis (`call_graph.png`, `req_graph.json`). Assim, a validade do produto final fica circunscrita à correção de um único passo não automatizável — o mapeamento `func_to_req`. Esta é uma força do desenho (o ponto de falha é isolado e auditável), mas também uma limitação: a verificação desse passo é, no presente trabalho, feita por inspeção do próprio investigador, sem confirmação independente (ver ponto seguinte).
- **Ausência de *inter-rater reliability*.** O protocolo `func_to_req` (Secção 3.5.2) é executado por um único investigador. Dois investigadores poderiam produzir mapeamentos diferentes, levando a grafos diferentes e a rankings potencialmente diferentes. A replicação por um segundo investigador, com cálculo de Cohen’s κ , é deixada como trabalho futuro.
- **Sensibilidade a parâmetros não testada sistematicamente.** O fator de amortecimento do PageRank ($d = 0,85$) e as tolerâncias de convergência (10^{-6} , 10^{-8}) seguem os valores canónicos da literatura. Em grafos com ≤ 17 arestas, a sensibilidade a estes parâmetros não é trivial — uma análise de robustez $d \in \{0,5, 0,7, 0,85, 0,95\}$ fica registada como extensão recomendada.
- **Critério “a priori” de convergência protege contra *p-hacking*.** A definição em Secção 3.4 foi formulada antes da observação dos resultados. Esta opção elimina o risco de o autor ajustar o critério *post hoc* para maximizar o número de pares “convergentes”.

4.6.3 Validade Externa (*External Validity*)

- **Dimensão e diversidade do corpus.** A validação assenta em três projetos Python; conclusões sobre a generalidade da abordagem a outros ecossistemas (Java, C++, TypeScript) reque-

rem extensão futura — nomeadamente, a substituição do *front-end* de AST por outro motor (JavaParser, TypeScript Compiler API ou libclang).

- **Seleção parcial de ficheiros.** Cada projeto foi analisado por meio de uma *amostra* dos seus ficheiros (2–4), selecionada como “*core*”. Esta seleção *introduz viés*: requisitos cujas implementações estão fora do conjunto selecionado tornam-se *singletons* (caso particularmente visível no Flask, com 6 requisitos isolados de 13). A replicação sobre o *codebase* completo de cada projeto fica como extensão recomendada.
- **Ausência de validação humana.** A coerência arquitetural foi avaliada por inspeção do investigador, não por engenheiros dos projetos analisados. Esta limitação é endereçada na Secção 5.2.

4.6.4 Validade de Conclusão (*Conclusion Validity*)

- **Tamanho amostral reduzido para inferências estatísticas.** Com $n = 7$ a 9 vértices, os coeficientes de correlação reportados têm intervalos de confiança largos; os valores devem ser lidos como *descritivos* dos três casos, não como base para inferência sobre uma população de projetos.
- **Observação parcial do Efeito TKC.** Os três grafos analisados, pela sua pequena dimensão e relativa regularidade de graus, não permitem observar o cenário em que o SALSA evidenciaria a sua vantagem teórica face ao HITS. A conclusão sobre o SALSA é, portanto, *honestamente nula*: o algoritmo não distinguiu o que os outros distinguiram, e a propriedade teórica não pôde ser empiricamente testada neste corpus.
- **Arestas não ponderadas.** Todas as arestas do Grafo de Requisitos têm peso unitário. Não é tida em conta a frequência de invocação ou a “força” semântica do acoplamento — fatores que potencialmente refinariam os *scores*.

Estas ameaças são endereçadas, sob a forma de propostas de continuação, na Secção 5.2.

Capítulo 5

Conclusão

O Capítulo 5 apresenta as principais conclusões da dissertação. A Secção 5.1 sintetiza as contribuições, revisita os objetivos enunciados em Secção 1.3 e regista o estado de execução de cada um. A Secção 5.2 identifica linhas de trabalho futuro.

Conceitos-chave: conclusão; síntese; contribuições; *research questions* respondidas; trabalho futuro.

5.1 Sumário e Principais Contribuições

Esta dissertação investigou a viabilidade de aplicar algoritmos de ranqueamento baseados em grafos — concretamente PageRank, HITS e SALSA — à priorização estrutural de requisitos de software. O problema original, exposto no Capítulo 1, partia da constatação de que as técnicas tradicionais de Priorização de Requisitos de Software (PRS), como a Votação Acumulativa, o AHP e o PERT/CPM, são fortemente dependentes da intervenção subjetiva dos *stakeholders* e não escalam para sistemas com elevado número de requisitos interdependentes (Achimugu et al., 2014; Bukhsh et al., 2020; Silva et al., 2018).

A resposta proposta consistiu em transferir parte do esforço de priorização para a topologia do sistema: se o grafo de dependências contém informação latente sobre a importância relativa dos requisitos, então um algoritmo de ranqueamento adequadamente escolhido pode extrair essa informação de forma reproduzível e automatizada. Para validar esta hipótese, foram desenvolvidos dois artefactos:

- **ReqGraph** — um protótipo Python, organizado como pacote instalável, que automatiza o *pipeline* Código-fonte $\rightarrow$ AST $\rightarrow$ Call Graph $\rightarrow$ Grafo de Requisitos. A análise é totalmente estática (não requer execução do código) e o protótipo gera artefactos em três formatos canónicos (PNG, JSON e DOT/Graphviz).
- `ranker.py` — um módulo de análise que aplica PageRank, HITS e SALSA aos grafos produzidos pelo *ReqGraph*, gerando *scores* comparáveis, gráficos de barras e relatórios consolidados. A partir de R5, o *script* `compute_rank_correlations.py` acrescenta ao *pipeline* o cálculo de Spearman ρ e Kendall τ_b entre cada par de algoritmos.

A validação foi realizada com **três projetos reais de código aberto** (CPython *stdlib*, Flask e *scikit-learn*), seleccionados como representantes de regimes crescentes de acoplamento arquitetural. Os principais resultados, detalhados no Capítulo 4, sustentam quatro contribuições:

1. **Validação da qualidade dos rankings extraídos automaticamente.** O *pipeline ReqGraph* produziu, sem intervenção humana adicional ao mapeamento `func_to_req`, grafos de requisitos coerentes com a arquitetura conhecida dos três projetos — incluindo a detecção emergente do ciclo `REQUEST_HANDLING ↔ CONTEXT ↔ ROUTING ↔ ERROR_HANDLING` no Flask e da centralidade de `REQ_PIPELINE` no *scikit-learn*. A validação realizada incidiu sobre a **qualidade dos resultados** (coerência com a arquitetura conhecida), e *não* sobre a sua adequação a percepções de profissionais — limitação assumida e encaminhada para trabalho futuro (Secção 5.2).
2. **Comparação empírica quantitativa entre PageRank, HITS e SALSA** sobre grafos reais de requisitos, com métricas formais de *rank correlation*. Os resultados (Tabela 4.5) mostram que a concordância entre algoritmos é *parcial e seletiva*: apenas dois dos 18 pares analisados satisfazem o critério forte de convergência ($\rho \geq 0,7$ e $\tau_b \geq 0,55$), ambos no caso mais esparsos (*stdlib*). Esta descoberta *contradiz* a hipótese inicial de que densidade implicaria convergência — e é, por si só, contribuição empírica relevante. A interpretação adequada é que cada algoritmo capta uma dimensão diferente de centralidade: PageRank capta importância transitiva, HITS *authority* capta dependência direta, HITS *hub* capta orquestração.
3. **Implementação manual e documentada do SALSA.** Dado que o NetworkX não fornece SALSA, foi desenvolvida uma implementação completa baseada em matrizes de transição de cadeias de Markov, com tratamento explícito de vértices absorventes. Esta implementação é reaproveitável em contextos académicos e industriais.
4. **Mecanismo de redução do custo de mapeamento.** O ficheiro `llm_prompt_mapping.md` formaliza um *prompt* estruturado para gerar o dicionário `func_to_req` via modelos de linguagem de grande escala, mitigando uma das fricções identificadas — a necessidade de mapear manualmente centenas de funções para domínios de requisito.

Estado dos objetivos e *research questions*. A Tabela 5.1 revisita os objetivos da Secção 1.3 e identifica o capítulo de resposta a cada uma das *research questions*.

Tabela 5.1 – Estado dos objetivos e das *research questions*.

Item	Estado	Capítulo / Secção
Objetivo (i) Formalização teórica	Concluído	Capítulo 2
Objetivo (ii) Protótipo <i>ReqGraph</i>	Concluído (<i>open source</i>)	Capítulo 3, Secção 3.2
Objetivo (iii) Avaliação empírica	Concluído sobre três casos	Capítulo 4
RQ1 (concordância entre algoritmos)	Respondida quantitativamente	Secção 4.3
RQ2 (efeito da densidade)	Respondida (<i>constraint intuitiva</i>)	Secção 4.3, 4.5
RQ3 (coerência arquitetural)	Respondida qualitativamente	Secção 4.5
Validação humana dos rankings	Não realizada (limitação)	Trabalho futuro (Secção 5.2)
Análise de vulnerabilidades topológicas	Apenas teórica	Capítulo 2, Secção 2.2.4
Modelo híbrido (vértices artificiais)	Não implementado	Trabalho futuro

A consistência entre as conclusões do referencial teórico e os resultados experimentais sustenta a

tese de que os algoritmos de ranqueamento estrutural constituem um **complemento** — não um substituto — às técnicas tradicionais de priorização, contribuindo para a **redução, não a eliminação**, do viés subjetivo em sistemas de larga escala. A subjetividade é deslocada, do julgamento de prioridade por *stakeholder* para a construção do mapeamento `func_to_req`, com vantagens operacionais que esta dissertação documenta empiricamente.

5.2 Trabalho Futuro

A investigação realizada abre várias linhas de continuação, ordenadas por proximidade ao trabalho desenvolvido e endereçando diretamente as ameaças à validade enunciadas em Secção 4.6.

- **Validação humana dos rankings.** A presente avaliação é estrutural e qualitativa. Um estudo controlado com engenheiros de software — produzindo rankings manuais sobre os mesmos projetos e comparando-os com os automáticos via correlação de Spearman ou Kendall — forneceria uma medida quantitativa direta de adequação. Esta validação foi conduzida por Silva et al. (2018) para o PageRank isoladamente; a sua replicação para o trio PageRank/HITS/SALSA, alargada a um corpus mais amplo, foi inviabilizada pela limitação de recursos disponível na presente investigação.
- **Inter-rater reliability do mapeamento `func_to_req`.** Replicar o protocolo (Secção 3.5.2) com um segundo investigador independente, reportando Cohen's κ e analisando o impacto das divergências de mapeamento sobre os rankings finais. Esta extensão endereça diretamente a ameaça à validade interna principal desta dissertação.
- **Modelo Híbrido com Vértices Artificiais.** Implementar a técnica de injeção de prioridades de negócio através de vértices artificiais conectados ao grafo de requisitos (Silva et al., 2018). Esta extensão permitirá medir empiricamente o impacto de prioridades subjetivas sobre os rankings estruturais e oferecer um mecanismo de ajuste fino aos *stakeholders*.
- **Análise de sensibilidade ao d do PageRank.** Sistematizar a robustez dos rankings face a $d \in \{0,5, 0,7, 0,85, 0,95\}$ e às tolerâncias de convergência, produzindo gráficos de estabilidade de *rank* por requisito.
- **Análise sobre *codebase* completo.** Reaplicar o *ReqGraph* aos três projetos sem seleção de ficheiros, reduzindo o número de *singletons* e permitindo observar se os rankings dos requisitos atualmente isolados (e.g. `REQ_BLUEPRINTS`, `REQ_TEMPLATING` no Flask) alteram a coerência arquitetural geral.
- **Extensão a outras linguagens.** O motor de extração de *Call Graph* baseia-se exclusivamente no módulo `ast` do Python. Estender o *ReqGraph* a Java (via `JavaParser`), TypeScript (via `TypeScript Compiler API`) ou C/C++ (via `libclang`) permitiria validar a generalidade da abordagem para além do ecossistema Python.
- **Integração contínua e *snapshots* temporais.** Aplicar o *ReqGraph* a *snapshots* sucessivos do mesmo projeto (e.g. *tags* de *releases*) permitiria estudar a **evolução estrutural** dos requisitos

ao longo do tempo, detetando *drift* arquitetural e regressões de acoplamento.

- **Tratamento de pesos semânticos.** Atualmente, todas as arestas têm peso unitário. Atribuir pesos derivados da frequência de invocação ou da força do acoplamento (e.g. número de funções partilhadas entre dois requisitos) poderá refinar substancialmente os rankings.

O conjunto destas linhas confirma que, embora o problema central tenha sido endereçado, a abordagem proposta abre um espaço de investigação largo no cruzamento entre Engenharia de Requisitos, Teoria dos Grafos e Análise Estática de Código.

Bibliografia

- Achimugu, P., Selamat, A., Ibrahim, R., & Mahrin, M. N. (2014). A Systematic Literature Review of Software Requirements Prioritization Research. *Information and Software Technology*, 56(6), 568–585.
- Ahl, V. (2005). *An Experimental Comparison of Five Prioritization Methods* [Master's thesis]. Blekinge Institute of Technology.
- Berander, P., & Andrews, A. (2005). Requirements Prioritization. In A. Aurum & C. Wohlin (Eds.), *Engineering and Managing Software Requirements* (pp. 69–94). Springer.
- Brin, S., & Page, L. (1998). The Anatomy of a Large-scale Hypertextual Web Search Engine. *Computer Networks and ISDN Systems*, 30(1–7), 107–117.
- Bukhsh, F. A., Bukhsh, Z. A., & Daneva, M. (2020). A Systematic Literature Review on Requirement Prioritization Techniques and Their Empirical Evaluation. *Computer Standards & Interfaces*, 69, 103389.
- Ding, C., He, X., Husbands, P., Zha, H., & Simon, H. D. (2002). PageRank, HITS and a Unified Framework for Link Analysis. *Proceedings of the 25th ACM SIGIR Conference*, 353–354.
- Firesmith, D. (2004). Prioritizing Requirements. *Journal of Object Technology*, 3(8), 35–48.
- Goodman, L. A., & Kruskal, W. H. (1954). Measures of Association for Cross Classifications. *Journal of the American Statistical Association*, 49(268), 732–764.
- Gotel, O., & Finkelstein, A. (1994). An Analysis of the Requirements Traceability Problem. *Proceedings of the First International Conference on Requirements Engineering*, 94–101.
- Gupta, P., Goel, A., Lin, J., Sharma, A., Wang, D., & Zadeh, R. (2013). WTF: The Who to Follow Service at Twitter. *Proceedings of the 22nd International Conference on World Wide Web (WWW '13)*, 505–514.
- He, X., Chen, T., Kan, M.-Y., & Chen, X. (2015). TriRank: Review-aware Explainable Recommendation by Modeling Aspects. *Proceedings of the ACM CIKM Conference*, 1661–1670.
- He, X., Gao, M., Kan, M.-Y., & Wang, D. (2014). BiRank: Towards Ranking on Bipartite Graphs. *IEEE Transactions on Knowledge and Data Engineering*, 26(11), 2673–2687.
- Jin, Y., Li, T., & Liu, S. (2009). Applying PageRank Algorithm in Requirement Concern Impact Analysis. *33rd Annual IEEE International Computer Software and Applications Conference*, 361–366.
- Karlsson, J. (2002). Software Requirements Prioritizing. *Proceedings of the Second International Conference on Requirements Engineering*, 110–116.
- Karlsson, J., & Ryan, K. (1997). A Cost-Value Approach for Prioritizing Requirements. *IEEE Software*, 14(5), 67–74.
- Karlsson, L., Berander, P., & Ågren, J. (2007). Pair-wise Comparisons versus Planning Game Partitioning: Experiments on Requirements Prioritisation Techniques. *Empirical Software Engineering*, 12(1), 3–33.

- Kendall, M. G. (1938). A New Measure of Rank Correlation. *Biometrika*, 30(1–2), 81–93.
- Kerzner, H. (2009). *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*. Wiley.
- Kleinberg, J. M. (1999). Authoritative Sources in a Hyperlinked Environment. *Journal of the ACM*, 46(5), 604–632.
- Leffingwell, D., & Widrig, D. (2003). *Managing Software Requirements: A Use Case Approach*. Pearson Education.
- Lempel, R., & Moran, S. (2001). SALSA: The Stochastic Approach for Link-structure Analysis. *ACM Transactions on Information Systems*, 19(2), 131–160.
- Li, F., & Yi, T. (2009). Apply PageRank Algorithm to Measuring Relationship's Complexity. *PACIIA 2008 Pacific-Asia Workshop on Computational Intelligence and Industrial Application*, 1, 914–917.
- Masuda, N., Porter, M. A., & Lambiotte, R. (2017). Random Walks and Diffusion on Networks. *Physics Reports*, 716–717, 1–58.
- Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.
- Ng, A. Y., Zheng, A. X., & Jordan, M. I. (2001). Stable Algorithms for Link Analysis. *Proceedings of the ACM SIGIR Conference*, 258–266.
- Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web* (rel. téc.). Stanford InfoLab.
- Rasiman, R., Dalpiaz, F., & Brinkkemper, S. (2022). How Effective Is Automated Trace Link Recovery in Model-Driven Development? *Requirements Engineering: Foundation for Software Quality*, 35–51.
- Ross, S. M. (2014). *Introduction to Probability Models* (11^a ed.). Academic Press.
- Saaty, T. L. (1980). *The Analytic Hierarchy Process*. McGraw-Hill.
- Silva, M. P., Tirelo, F., & Marques Neto, H. T. (2018). Uso do Algoritmo PageRank para Priorização de Requisitos de Software. *Anais do Congresso Brasileiro de Software: Teoria e Prática*.
- Sommerville, I. (2007). *Engenharia de Software* (8^a ed.). Pearson.
- Spearman, C. (1904). The Proof and Measurement of Association between Two Things. *The American Journal of Psychology*, 15(1), 72–101.
- Szwarcfiter, J. L. (2018). *Teoria Computacional de Grafos*. Elsevier.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.
- Zhou, D., Bousquet, O., Lal, T. N., Weston, J., & Schölkopf, B. (2004). Learning with Local and Global Consistency. *Advances in Neural Information Processing Systems*, 321–328.

Glossário

Termo	Definição
AHP	<i>Analytic Hierarchy Process</i> . Método de priorização proposto por Saaty (1980), baseado em comparações aos pares ponderadas por níveis hierárquicos de critérios; tem complexidade $O(n^2)$ no número de itens.
AST (<i>Abstract Syntax Tree</i>)	Representação em árvore da estrutura sintática de um programa, usada como ponto de partida para análise estática de código-fonte.
Call Graph	Grafo direcionado em que cada vértice é uma função (ou método) e cada aresta (f_i, f_j) indica que f_i invoca f_j .
Cadeia de Markov	Processo estocástico em que a probabilidade do estado seguinte depende apenas do estado atual. Subjacente aos três algoritmos analisados nesta dissertação.
Dangling Node	Vértice sem arestas de saída. No PageRank é tratado por redistribuição uniforme da sua massa de probabilidade, garantindo ergodicidade da cadeia de Markov.
Distribuição estacionária	Vetor probabilístico $\mathbf{r}^*$ que verifica $\mathbf{r}^* = M\mathbf{r}^*$, isto é, é invariante sob aplicação da matriz de transição M . Existe e é única em cadeias de Markov ergódicas.
Efeito TKC	<i>Tightly Knit Community Effect</i> : vulnerabilidade do HITS em que subcomunidades densas concentram desproporcionalmente os <i>scores</i> ; o SALSA foi projetado para mitigá-lo.
Ergodicidade	Propriedade de uma cadeia de Markov que garante a existência de uma distribuição estacionária única, independente do estado inicial.
Feature module	Agregação de funções e métodos do código relacionada a uma capacidade funcional do sistema; usado nesta dissertação como <i>proxy</i> para “requisito”.
Grafo de Requisitos	Grafo direcionado derivado do <i>Call Graph</i> através de um mapeamento <code>func_to_req</code> , em que cada vértice é um requisito de domínio e as arestas representam dependências entre requisitos.
HITS	Algoritmo de Kleinberg (1999) que atribui a cada vértice dois <i>scores</i> : <i>hub</i> (orquestrador) e <i>authority</i> (dependência central).
PageRank	Algoritmo de ranqueamento que modela a probabilidade estacionária de um navegador aleatório visitar cada vértice do grafo (Brin & Page, 1998; Page et al., 1999).

Termo	Definição
Kendall τ_b	Coeficiente de correlação de ranks baseado em pares concordantes e discordantes, com correção para empates (Goodman & Kruskal, 1954; Kendall, 1938). Toma valores em $[-1, 1]$.
<i>Rank Correlation</i>	Medida da semelhança entre duas ordenações; nesta dissertação é utilizada para comparar rankings produzidos por diferentes algoritmos.
<i>Random Walk</i>	Processo estocástico que descreve a trajetória de um caminhante que se move aleatoriamente entre vértices de um grafo, seguindo probabilidades associadas às arestas.
<i>SALSA</i>	Algoritmo de Lempel e Moran (2001) que combina HITS com cadeias de Markov sobre um grafo bipartido, normalizando localmente por grau.
Spearman ρ	Coeficiente de correlação de Pearson aplicado a ranks; mede a força e direção da relação monotônica entre duas ordenações (Spearman, 1904). Toma valores em $[-1, 1]$.
<i>Threats to validity</i>	Conjunto de ameaças à generalização e à interpretação de um estudo empírico, organizadas por Wohlin et al. (2012) em quatro categorias: <i>construct, internal, external, conclusion</i> .

Apêndice ou Anexo?

O Apêndice 5.2 explica a diferença entre apêndice e anexo, conforme as convenções do *template* oficial da Universidade Lusófona.

Apêndice. Apêndices englobam materiais elaborados pelo autor(a), tais como gráficos, quadros, tabelas, traduções, organogramas e esquemas que prestem informação relevante para a compreensão do trabalho. Só devem figurar nos apêndices as informações previamente referenciadas no texto. As informações são total ou parcialmente da responsabilidade do autor.

Anexo. Anexos englobam documentos que, não sendo elaborados pelo autor, serviram de base para a construção do estudo ou facilitam a compreensão da tese/dissertação. Só devem figurar nos anexos documentos e/ou materiais previamente referenciados no corpo do trabalho. Todos os documentos devem estar em formato digital.

Cálculo de Spearman e Kendall em Python puro

Este apêndice apresenta a implementação completa do *script* `compute_rank_correlations.py` (Capítulo 3, Secção 3.4), em Python puro e sem dependência de `scipy`. O *script* consome os ficheiros `ranking_results.json` produzidos pelo `ranker.py` e gera o ficheiro consolidado `rank_correlations.json`, utilizado para popular a Tabela 4.5.

```
def mean_ranks(scores):
    """Converte scores em ranks com mean-rank para ties."""
    indexed = sorted(enumerate(scores), key=lambda x: -x[1])
    ranks = [0.0] * len(scores)
    i = 0
    while i < len(indexed):
        j = i
        while j + 1 < len(indexed) and indexed[j+1][1] == indexed[i][1]:
            j += 1
        avg = (i + 1 + j + 1) / 2.0
        for k in range(i, j + 1):
            ranks[indexed[k][0]] = avg
        i = j + 1
    return ranks

def spearman_rho(a, b):
    return pearson(mean_ranks(a), mean_ranks(b))

def kendall_tau_b(a, b):
    """Kendall tau-b com correcao para empates (Goodman, 1954)."""
    n = len(a); C = D = Ta = Tb = 0
    for i in range(n):
        for j in range(i + 1, n):
            da, db = a[i] - a[j], b[i] - b[j]
            if da == 0 and db == 0: Ta += 1; Tb += 1
            elif da == 0: Ta += 1
            elif db == 0: Tb += 1
            elif (da > 0) == (db > 0): C += 1
            else: D += 1
    P0 = n * (n - 1) / 2
    den = ((P0 - Ta) * (P0 - Tb)) ** 0.5
    return (C - D) / den if den else float('nan')
```

Listing 1 – Implementação de Spearman ρ e Kendall τ_b em Python puro (excerto).

A versão completa, incluindo o carregamento dos ficheiros JSON e a escrita do resultado consolidado, encontra-se em `compute_rank_correlations.py` na raiz do repositório público.

Mapeamentos `func_to_req` dos três casos de estudo

Os mapeamentos `func_to_req` utilizados em cada caso de estudo encontram-se em:

- CPython *stdlib*: `testes/simples_stdlib/mapeamento.py`
- Flask: `testes/medio_flask/mapeamento.py`
- *scikit-learn*: `testes/complexo_sklearn/mapeamento.py`

Os ficheiros são distribuídos como parte do repositório público mencionado em Capítulo 1, Secção 1.4. Cada ficheiro contém um único dicionário Python `FUNC_TO_REQ` cujas chaves são nomes qualificados de funções/métodos e cujos valores são identificadores de requisitos no formato `REQ_*`.